

1. INTRODUCTION

1.1 OVERVIEW OF THE SYSTEM

Modern pet care may or may not be simple depending upon the pet breed and health conditions. Caring for dogs, cats, and specialized companion animal breeds are demanding tasks that require close attention to schedules, nutrition, and early physical signs.

A pet health and breed identification system is a tool designed to help pet owners and veterinary administrators record, identify, and manage pet profiles. Utilizing image classification models and rule-based assessment pipelines, it offers an intuitive way to identify breeds and review health issues.

This system calculates basic health risk indexes, highlighting skin, fur, and weight concerns. Users upload a photo of their pet for breed identification and submit close-up images to perform preliminary health scans. It provides veterinary advisory content, including customized vaccination records, nutrition guides, and booking appointments with veterinarians. Whether dealing with a skin issue or searching for a clinical professional, our platform provides guidance.

The project is divided into Accounts, Pets, Records, Advisory, and Analytics modules. Accounts manages the owner profiles, login, and registration. Pets handles pet profile storage, breed predictions, and health assessments. Records manages vaccinations, treatment details, and reminders. Advisory manages catalogs of safe medicines and clinical providers. Analytics provides visual dashboards of pet records.

1.2 PROBLEM DEFINITION AND OBJECTIVE OF THE PROJECT

Pet care platforms help individuals and organizations manage pet health in multiple ways, including:

- o **Profile Management:** Owners can register and store details such as name, species, breed, and weight to trace their pets' identity over time.
- o **Vaccination Tracking:** Pet care workers can log completed vaccines and view scheduled doses to protect pets from infectious diseases.
- o **Health Assessment:** System-generated rules flag risk indicators, providing owners with early warning guidance.
- o **Service Booking:** Registered users can discover veterinary clinics, groomers, or trainers, and book appointments directly.

Pet health and breed identification systems resolve the problems of scattered medical history. Owners often struggle to locate vaccination cards, lose track of medication reminders, and lack direct guidance on breed-specific traits. By keeping a digital diary, users ensure accurate information is available whenever a veterinary visit is planned.

Common metrics tracked in the system include:

- Identification: Image-based breed classification, confidence scores, and species check
- Health Risks: Skin infection risk, fur loss risk, eye issues, and obesity index
- Medical Logs: Vaccines, clinic visits, prescriptions, and follow-up dates
- Schedules: Vaccination timelines, grooming intervals, and appointment status

2. SYSTEM ANALYSIS

2.1 INTRODUCTION

System analysis is a key phase of the software development life cycle. The current problem environment is investigated in detail. The system analyst plays the role of an investigator, tracing the operational flow, processing steps, inputs, outputs, and stakeholder constraints. The objective is to design a digital system that replaces manual workflows and scattered information files.

2.2 IDENTIFICATION OF NEED

The work associated with software engineering is divided into analysis, design, development, and testing phases. The system requirement analysis is conducted to determine what functions the platform must perform. The need is to establish a secure, centralized pet-care database that assists owners in identifying breeds, tracking vaccinations, and booking clinical services efficiently, reducing administrative paper overhead.

2.3 EXISTING SYSTEM

In the existing pet care environment, records are kept in a scattered manner. Owners rely on paper booklets issued by clinics, manual notes, and scattered web articles for guidance. This manual process is unorganized and makes tracking records difficult. There is no integrated breed classifier or automated email reminder schedule.

2.3.1 LIMITATION OF EXISTING SYSTEM

The existing manual methods have several limitations compared to the proposed system:

1. Missing reminders: Owners often forget booster vaccination dates due to lack of notifications.
2. Search barriers: Finding available clinics and booking appointments requires manual coordination.
3. Breed uncertainty: Adopted pet owners lack immediate information on breed characteristics.

2.4 PROPOSED SYSTEM

The proposed system is a centralized Django web application that stores pet profiles, executes AI breed identification, conducts preliminary health scans, maintains medical histories, schedules reminders, and supports appointment bookings. It provides owners with a digital care dashboard and administrators with a staff management control screen.

2.4.1 ADVANTAGES OF PROPOSED SYSTEM

1. Centralized records: Vaccination logs, medical history, and appointments are stored in one database.
2. AI-assisted guidance: Image-based breed classification and health scans provide immediate feedback.
3. Automated scheduling: Reminders are generated automatically from record dates and sent to owners.

2.5 FEASIBILITY STUDY

Feasibility study is essential to determine whether the identified requirements can be satisfied within technological, operational, economic, and time constraints. A preliminary study ensures the proposed platform is practical to develop and implement before resources are committed.

2.5.1 TECHNICAL FEASIBILITY

The technical feasibility evaluates the required hardware and software. The platform uses Python 3.10+, Django 5.2, SQLite3, and TensorFlow/Keras libraries. Since these are standard open-source tools with active developer support, the system is technically feasible.

2.5.2 ECONOMIC FEASIBILITY

The economic feasibility study determines if the system's benefits justify its cost. Using open-source frameworks eliminates licensing expenses. The project is highly cost-effective, requiring only standard web hosting and zero licensing fees.

2.5.3 OPERATIONAL FEASIBILITY

Operational feasibility determines if the system will be accepted and utilized by the target audience. The platform features responsive web templates, simple navigation, and clear form buttons. Administrators can manage services and update records through standard Django views. The platform is operationally feasible.

2.5.4 SCHEDULE FEASIBILITY

Schedule feasibility measures the project completion timeline. Staged development divides the project into authentication, pet profiling, ML integration, medical records, and analytics dashboards, ensuring completion within schedule.

2.6 SYSTEM SPECIFICATION

System requirements are systematically organized in the form of an SRS document. It serves as a contract between the developers and clients, ensuring the system operates exactly as specified.

2.6.1 SOFTWARE SPECIFICATION

A software requirements specification is a description of a software system to be developed. It establishes the baseline for the project design, database structure, and testing scenarios.

- **HTML:** It is the standard mark-up language for documents designed to be displayed in a web browser. It describes the structure of a web page semantically. Browsers do not display the tags but use them to interpret the content of the page. In this platform, HTML elements form the basis of the user-facing templates.

Front End (Continued)

- **Cascading Style Sheets (CSS) / Bootstrap:** CSS defines the styling, margins, colors, and responsive formatting of web documents. Bootstrap is a mobile-first, front-end framework utilized in this project to create polished interface cards, navigation bars, and form validation styles with minimal custom code.

Back End

- **Python / Django:** Python is a high-level, general-purpose language characterized by readability and rich scientific library ecosystems. Django is an MTV (Model-Template-View) framework that encourages rapid, secure backend development.

Django manages database sessions, cookie authorization, form verification, and route mappings. It coordinates SQL interactions using the Object-Relational Mapper (ORM), handling queries, updates, and deletes without requiring raw SQL command injection. This framework coordinates inputs, routes requests, processes image files, and integrates machine learning classifiers seamlessly.

Back End (Continued)

• Django Framework Features:

1. MVT Architecture: Separation of database schemas (Models), visual layouts (Templates), and control logic (Views).
2. Built-in Security: Automatic protection against cross-site scripting (XSS), SQL injection, and CSRF attacks.
3. Django ORM: Interacting with database rows as Python model instances, ensuring portability across database engines.
4. User Authorization: Group-based and staff-only login controls, facilitating secure role separation.

• Django Framework Advantages:

1. Rapid prototyping: Pre-configured components for authentication, forms, and session management.
2. Community support: Vast array of packages and comprehensive guides for machine learning integration.
3. Scalability: Handles high traffic and can scale from SQLite3 to PostgreSQL with minor configuration changes.

• Django Framework Disadvantages:

1. Monolithic configuration: May introduce unnecessary overhead for simple single-page web applications.
2. Complex routing: The MVT pattern has a learning curve for developers unfamiliar with full-stack frameworks.

In summary, Python and Django provide a highly secure, reliable, and maintainable foundation for full-stack pet care platforms.

Database and Frameworks

- **SQLite3 Database:** SQLite is a serverless, self-contained relational database management system. It writes directly to a single disk file, making it lightweight and highly suitable for development. The platform uses SQLite3 to store pet records, vaccinations, bookings, and medical history logs.
- **TensorFlow / Keras:** TensorFlow is an open-source machine learning framework. Keras provides high-level APIs for defining neural networks. The project uses a pre-trained CNN model loaded locally using Keras to classify dog and cat breeds from uploaded images.
- **Operating System:** Windows 10/11 or Linux provides the operating runtime environment, managing file systems, media directory creations, and script executions.

2.6.2 TOOLS SPECIFICATION

Tools: Google Chrome, Visual Studio Code, Python CLI, SQLite Database Browser

2.6.3 HARDWARE SPECIFICATION

Processor: Intel Core i3 or above (dual-core minimum)

RAM: 4GB minimum (8GB recommended for ML inference)

Hard Disc: 2GB minimum for model files and database

Display Type: PC Display / Mobile Display

Input Device: Mouse, Keyboard, Camera (for pet image capture)

Output Device: Monitor, Mobile Display

3. SYSTEM DESIGN

3.1 INTRODUCTION

System design is the process of translating requirements into a technical plan for software development. This phase maps data flows, entity relationships, use cases, input forms, and output screens. The blueprint ensures that the code matches the user requirements in a secure, efficient, and maintainable structure. Key design considerations include usability, database integrity, process separation, and data validation.

3.2 INPUT DESIGN

Input design focuses on converting user inputs into a structured database format. Inaccurate inputs are the most common source of data processing errors. In this system, user inputs include owner registration forms, profile details, pet profile forms, breed identification image uploads, health scan files, vaccination logs, medical record detail entries, and service bookings. Validation rules ensure required fields are not blank and only valid image formats under 5MB are accepted.

3.3 OUTPUT DESIGN

Computer output is the source of processed information for users. Efficient output design ensures information is presented in a clear, understandable format. System outputs include breed prediction labels with confidence scores, health risk assessment summaries, active reminders, search results for local clinics and doctors, booking status confirmations, and dashboard analytics charts.

3.4 DATABASE DESIGN

Database design defines how information is structured in persistent tables. The platform organizes schemas to prevent data redundancy and protect relationship integrity. Tables are linked using foreign keys, such as referencing the owner in Pet tables and referencing the pet in Vaccination records, ensuring cascaded updates.

3.4 DATABASE DESIGN (Continued)

Normalization organizes tables to avoid redundancies and prevent insertion, update, and deletion anomalies. The schemas are designed to comply with Third Normal Form (3NF):

- **First normal form (1NF):** Each table attribute holds only atomic values. Multi-valued lists are split into separate rows. For example, vaccine types are stored as unique records.
- **Second normal form (2NF):** The table must comply with 1NF, and all non-prime attributes must depend entirely on the primary key, eliminating partial dependencies.
- **Third normal form (3NF):** The table complies with 2NF, and no non-prime attribute depends transitively on the primary key. All transitive dependencies are removed.

By enforcing 3NF, the relational schema minimizes storage overhead and ensures consistency across user and pet profiles. For example, a PetSighting record refers to a MissingPet primary key, and the MissingPet refers to a User primary key, preventing duplicated owner details in sighting records.

Below is a summary of how normalization is applied to key models in the pet care system:

1. **1NF:** Phone number, address, and city in UserProfile are stored in separate atomic columns.
2. **2NF:** Pet records depend entirely on the auto-generated primary key ID, which uniquely identifies each animal.
3. **3NF:** MedicalRecord visit date and diagnosis depend only on the record ID and not transitively on the user ID.

3.4 DATABASE DESIGN (BCNF Application)

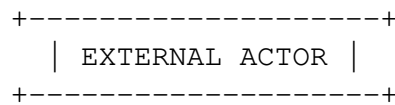
Boyce-Codd Normal Form (BCNF) is an advanced version of 3NF, requiring that for every non-trivial functional dependency $X \rightarrow Y$, X must be a super key. In our database models (UserProfile, Pet, BreedPrediction, HealthAssessment, VaccinationRecord, MedicalRecord, BreedInfo, DiseaseInfo, VetDoctor), all attributes are fully dependent on their respective primary keys, preventing anomalies.

3.5 DATA FLOW DIAGRAM (DFD)

The Data Flow Diagram (DFD) represents the flow of information through processes, data stores, and external entities. It uses a small set of primitive symbols to detail how input data is processed into outputs.

BASIC DFD SYMBOLS

External Entity Symbol: An external actor (User, Admin, or Visitor) is represented using a rectangle. These actors provide inputs to the processes or consume outputs generated by the system.



Process Symbol: Functions or processes that transform data are represented using circles. They are labeled with the function name.

BASIC DFD SYMBOLS (Continued)

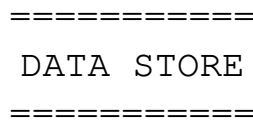
Process Symbol: The functions are represented using circles. Bubbles are annotated with the names of the corresponding functions. They convert data into information.



Data Flow Symbol: A directed arc or an arrow is used as a Data Flow Symbol. This represents the data flow occurring between two processes or between an external entity and a process in the direction of the Data Flow Arrow. Data flow symbols are annotated with corresponding data names.

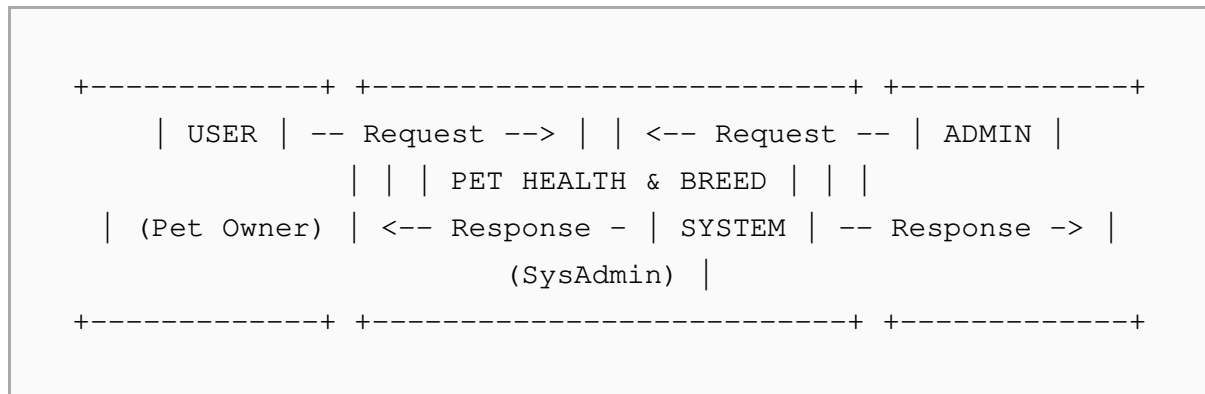


Data Store Symbol: A Data Store represents a logical file. It is represented using two parallel lines. A logical file can represent either a data structure or a physical file on disk. Each data store is connected to a process by means of a Data Flow Symbol.



DFD FOR PET HEALTH & BREED SYSTEM

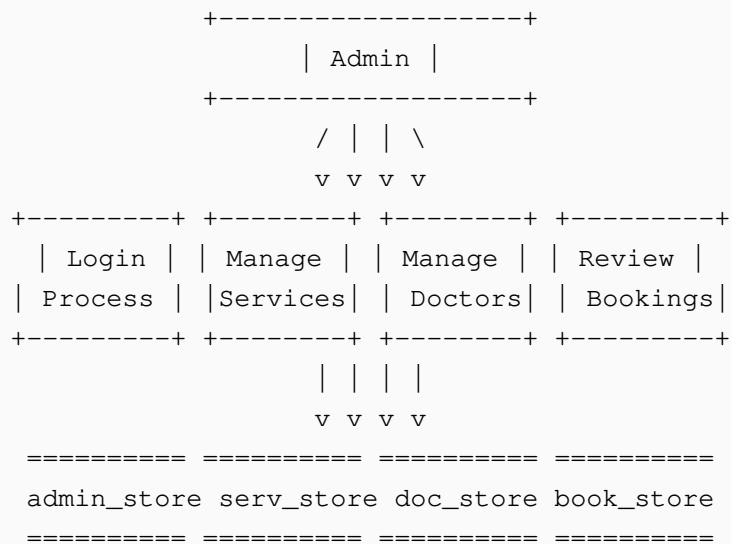
Level 0



The Level 0 DFD provides a high-level conceptual overview of the system. It maps the central process bubble interacting with two primary external entities: the User (Pet Owner) and the Admin (System Administrator). The User inputs registration details, pet profile forms, breed prediction photos, and clinic booking parameters, receiving health risk charts and schedules. The Admin manages clinical services, doctor lists, and system catalogs, and reads contact forms and reviews.

DFD FOR PET HEALTH & BREED SYSTEM

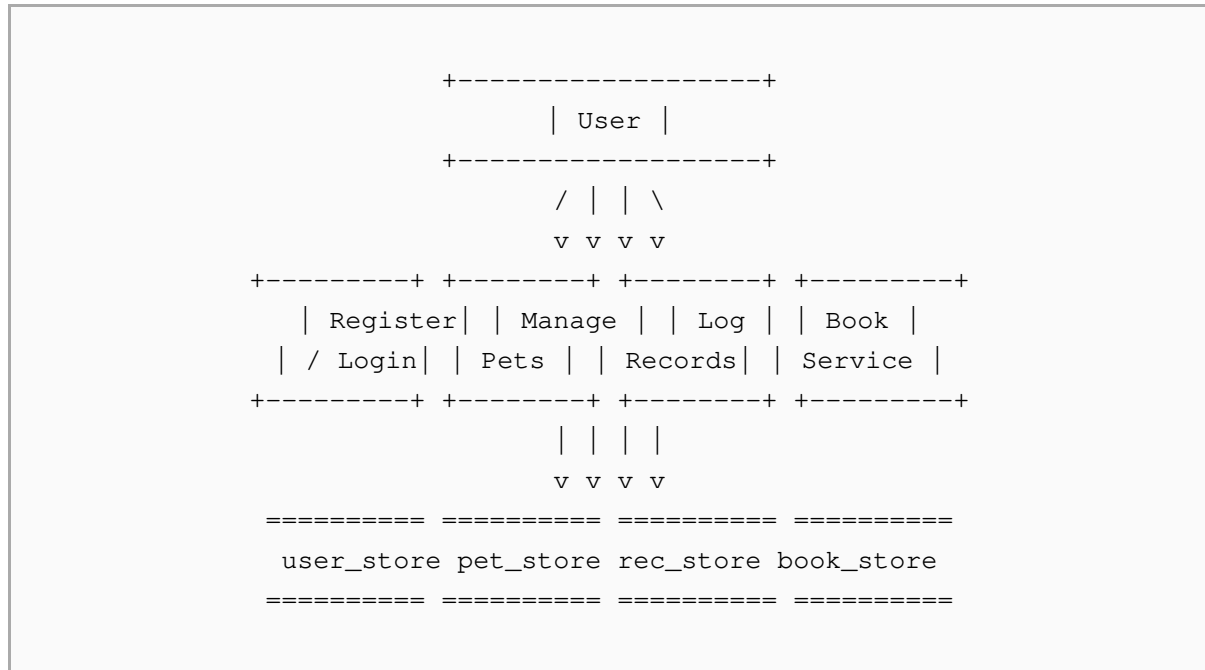
Level 1 - Admin



The Level 1 DFD for the Admin entity details administrative operations. The Admin logs in to access restricted views, updates pet service locations, adds veterinary doctors, and modifies booking states (e.g. confirming appointment requests). Changes are saved to their respective data stores, which are then read by users searching for local services.

DFD FOR PET HEALTH & BREED SYSTEM

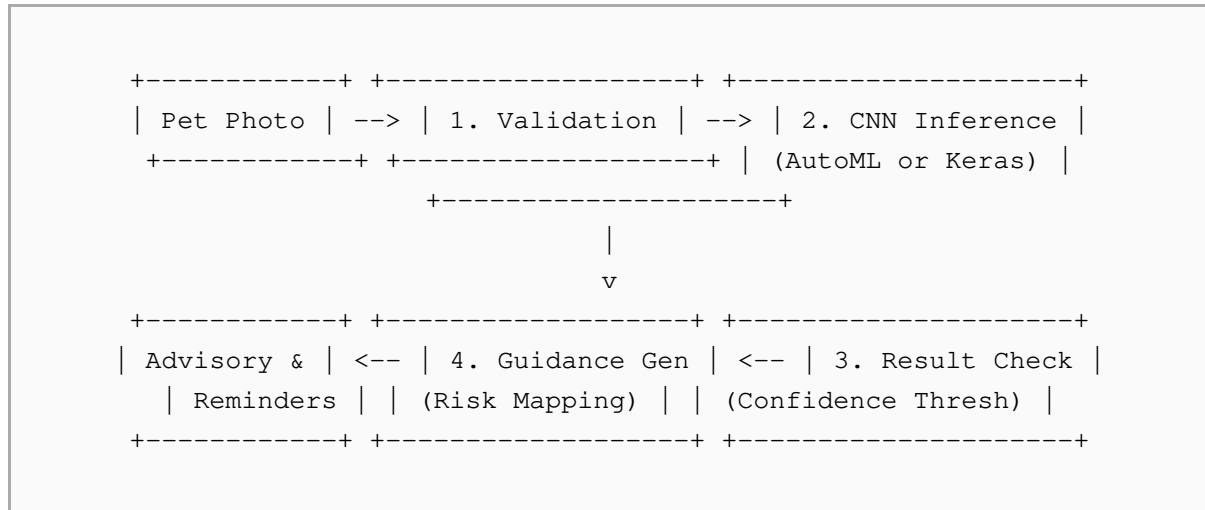
Level 1 - User



The Level 1 DFD for the User entity traces owner-facing processes. The User registers or authenticates, creates pet profile records, registers vaccinations and medical history entries, and books veterinary clinic services. The actions are saved to user, pet, medical record, and booking stores respectively, enabling comprehensive history retrieval.

DFD FOR PET HEALTH & BREED SYSTEM

Level 1 - AI Workflows

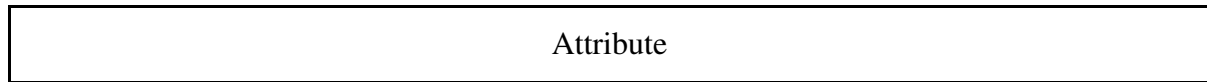


The Level 1 DFD for AI workflows details breed prediction and health assessment processes. The user uploads a photo, which is validated for format and size. The validated image is processed by the local prediction function running CNN model files. If the breed classification confidence falls below 50%, a fallback status is flagged. Results are linked to pet records, and preliminary care risk levels are calculated.

3.6 ENTITY RELATIONSHIP DIAGRAM

Entity-Relationship Diagram (ERD) defines the logical schema of the system database, mapping entities, their attributes, and their relationships. Rectangles represent entities, ovals represent attributes, and diamonds represent relationships.

Attributes: Attributes are properties that define an entity type, represented by ovals in the ER diagram.



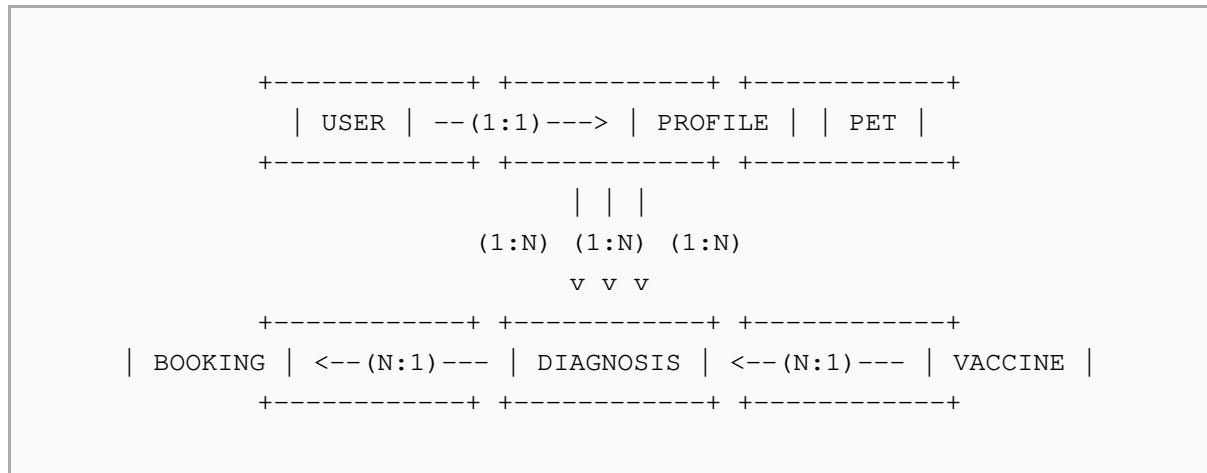
Relationship: Associations between entity types are represented using diamond shapes.



Entity: A basic data object is represented as a rectangle in an ER diagram.



ENTITY RELATIONSHIP DIAGRAM



The Entity Relationship Diagram (ERD) defines the relational structure of the database. A User has a one-to-one relationship with a UserProfile and a one-to-many relationship with Pets. A Pet record has one-to-many relationships with BreedPrediction, HealthAssessment, VaccinationRecord, and MedicalRecord tables. A ServiceBooking links a User, a Pet, and optionally a VetDoctor, establishing relationship integrity across the booking process.

3.7 USE CASE DIAGRAM

A use case diagram is a graphical depiction of a user's possible interactions with a system, detailing the relationships between external actors and system boundaries. The use cases are represented by horizontally shaped ovals.

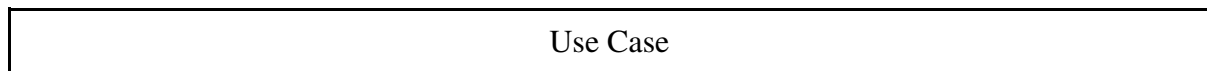
Actor: The users that interact with a system. An actor can be a person, an organization, or an outside system that interacts with your application or system. They must be external objects that produce or consume data.



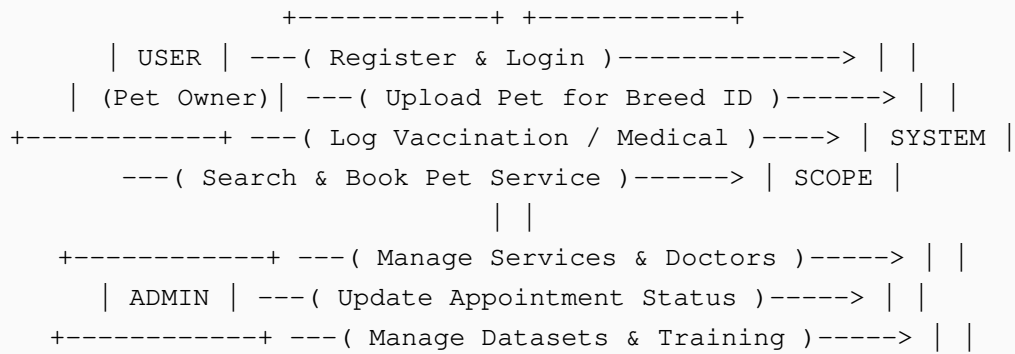
System: A box that sets a system scope to use cases. All use cases outside the box would be considered outside the scope of that system.

Association: A line between actors and use cases. In complex diagrams, it is important to know which actors are associated with which use cases.

Use cases: Horizontally shaped ovals that represent the different uses that the system might have.



USE CASE DIAGRAM



The Use Case Diagram defines interactions between actors and the system. The User (Pet Owner) actor executes use cases for registration, pet profile updates, breed prediction, health scans, logging vaccinations, creating reminders, finding services, and requesting bookings. The Admin actor manages service provider listings, vet doctor schedules, confirms booking requests, reviews contact forms, and uploads training datasets.

4. SYSTEM DEVELOPMENT

4.1 MODULE DESCRIPTION

The Pet Health and Breed Identification System is developed as a modular web application containing several key functional units:

o Accounts Module: Manages user profiles, registration, login, and OTP-based passwordless reset. It secures pages and stores contact details.

o Pets Module: Coordinates pet profile creation, stores species and weight, and handles breed identification and health assessment scan uploads.

o Records Module: Records vaccination schedules (doses completed, dates, clinical details) and medical history (clinic visit dates, diagnoses, treatments, prescriptions).

o Advisory Module: Manages reference catalogs of veterinary services, available doctors, CareAdvisory advice templates, and safe MedicineRecommendations.

o Analytics Dashboard Module: Formulates data queries via Django ORM and generates responsive visual summaries of owner records and platform metrics.

o Admin ML Module: A staff interface for uploading dataset ZIP files, displaying class distributions, triggering model training, and confirming bookings.

Each module maps to distinct URL routes and database tables, ensuring a loose coupling that facilitates independent maintenance and updates.

4.2 AI BREED & HEALTH PIPELINE

The AI Breed Identification and Health Assessment pipeline coordinates the core machine learning inference and risk mapping procedures. When an owner uploads an image, the system validates the file and executes prediction workflows:

1. **Image Preprocessing:** Images are loaded using the Python Image Library (PIL) and resized to 224x224 pixels, corresponding to standard MobileNetV2 inputs, with pixel normalization.
2. **AutoML / Vertex AI vision client:** A pluggable client connects to Vertex AI endpoints for cloud classification. A fallback local prediction runner loads weight files locally when cloud API rates are limited.
3. **Prediction Storage:** The predicted breed, species, confidence score, and model version are saved in the BreedPrediction table, linked to the Pet record.
4. **Health Risk Mapping:** In health scans, the model checks for symptoms of skin infections, fur loss, wounds, eye issues, and obesity, mapping detections to Low, Medium, or High risk levels, and generates care recommendations.

This design separates the ML logic from database operations, allowing weight files or API models to be upgraded without modifying user views or database schemas.

5. SYSTEM IMPLEMENTATION

5.1 TESTING

Software testing involves executing individual components to identify bugs or logical errors. The goal is to verify that the application operates reliably and matches all requirements.

UNIT TESTING

Unit testing validates that individual code units perform as designed. In Django, views, models, and forms are tested using test cases. For example, testing the PetForm validates that name and species are required, while testing the UserProfile model validates that a profile is created when a user registers.

INTEGRATION TESTING

Integration testing checks that modules operate together consistently. The platform tests the link between database records. For instance, integration tests check that adding a scheduled vaccination automatically generates an active record in the Reminder data store with the correct due date.

SYSTEM TESTING

System testing validates the complete application workflow. End-to-end test scenarios verify user registration, login, pet profile creation, breed image prediction uploads, booking requests, and administrative status updates.

5.1 TESTING (Continued)

REGRESSION TESTING

Regression testing ensures that code changes or updates do not introduce side effects in existing features. A test suite is executed after every change to verify that login checks, profile edits, and record filters continue to operate correctly.

5.2 VALIDATION CHECK

Validation ensures the platform accepts only correct and structured inputs. Form validation blocks empty fields, verifies email structures, ensures passwords are strong, and validates date formats. Media validation enforces that uploaded files are valid image extensions (JPG, PNG) and are within size limits (under 5MB) to protect system memory resources.

Key validation rules applied to form entries in the platform include:

- **File Format Check:** Rejects non-image files for pet profiles or scans, displaying error messages.
- **Date Limit:** Restricts booking dates to current or future dates, preventing past entries.
- **Numeric Range:** Restricts pet weight to positive values, showing validation errors for invalid numbers.

5.3 SYSTEM IMPLEMENTATION

System implementation coordinates the deployment of the application in actual use. It includes setting up database tables, configuring environment files, collecting static resources, and verifying server configurations.

Key deployment steps include:

1. **Database Migration:** Django migrations are executed using the python command-line interface to create relational database tables in the SQLite3 file.
2. **Environment Configuration:** Security credentials, secret keys, debug flags, and media directory paths are set up inside the system environment files.
3. **Static Resource Collection:** Bootstrap stylesheets, custom CSS styles, Javascript files, and icons are gathered into a single static directory for browser serving.

Once migrations, environment parameters, and static directories are configured, the web application runs on the target development server, verifying that routing, SQL connections, and image uploads function properly.

6. SYSTEM MAINTENANCE AND FUTURE ENHANCEMENT

6.1 SYSTEM MAINTENANCE

System maintenance is required to ensure the long-term reliability, security, and performance of the web application. It includes regular tasks to prevent database issues and protect system resources:

- **Database Backups:** Weekly database backups are performed to copy the SQLite3 file, preventing data loss in case of hardware failures.
- **Software Updates:** Periodic package updates are run to upgrade Django, PIL, and TensorFlow dependencies, applying security patches.
- **Performance Monitoring:** Server logs and transaction tables are monitored to identify slow queries and improve index configurations.

6.2 FUTURE ENHANCEMENTS

The platform is designed with an extensible modular structure, allowing future upgrades:

1. **Mobile Application:** Building Android and iOS clients that connect to Django views through REST APIs.
2. **Veterinary Chat:** Integrating real-time messaging between pet owners and registered veterinary doctors.
3. **Wearable Integration:** Syncing the pet care dashboard with GPS collars or activity monitors to track pet metrics automatically.

7. CONCLUSION

7.1 CONCLUSION

The Pet Health and Breed Identification System has been developed as an integrated web application built using Python, Django, SQLite3, Bootstrap, and machine learning components. The platform provides pet owners with a unified system to manage pet profiles, identify breeds, perform preliminary health scans, log vaccinations, keep medical histories, schedule reminders, book appointments, and report missing pets.

The application follows standard software engineering design principles, separating logic into loose apps (Accounts, Pets, Records, Advisory, Analytics) and applying 3NF database design. Security measures include CSRF protection, form validation, password hashing, and role-based access controls. The AI breed classifier and health assessment pipeline offer immediate guidance, helping owners identify issues early.

In academic evaluation, this project demonstrates full-stack web application development, database design, ML library integration, validation checks, and testing. It addresses the real-world problem of scattered pet records, helping owners organize information and make informed care decisions. Future enhancements, including mobile applications and real-time chat, will further increase the platform's value.

8. APPENDIX

8.1 APPENDIX A. TABLE DESIGNS

1. UserProfile

Table name: accounts_userprofile | Description: User details | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
user_id	int (FK)	Foreign key to auth_user (OneToOne)
phone_number	varchar (20)	Owner phone number
address	varchar (255)	Street address
city	varchar (100)	City of residence
country	varchar (100)	Country of residence
created_at	datetime	Profile creation timestamp

2. Pet

Table name: pets_pet | Description: Pet profile details | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
owner_id	int (FK)	Foreign key to auth_user (ForeignKey)
name	varchar (100)	Pet name
species	varchar (20)	Species choice (dog, cat)
breed	varchar (100)	Pet breed name
date_of_birth	date	Pet date of birth
weight_kg	decimal (5,2)	Pet weight in kilograms
image	varchar (file)	Pet image file path
created_at	datetime	Record creation timestamp

8.1 APPENDIX A. TABLE DESIGNS (Continued)

3. BreedPrediction

Table name: pets_breedprediction | Description: Breed predictions | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
pet_id	int (FK)	Foreign key to pets_pet
predicted_breed	varchar (100)	Predicted breed label
species	varchar (20)	Species class predicted
confidence	decimal (5,2)	Prediction confidence percentage
model_version	varchar (50)	Classifier model version
created_at	datetime	Inference timestamp

4. HealthAssessment

Table name: pets_healthassessment | Description: Health risk assessments | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
pet_id	int (FK)	Foreign key to pets_pet
assessment_date	datetime	Scan execution timestamp
skin_infection_risk	varchar (10)	Skin infection risk level
fur_loss_risk	varchar (10)	Fur loss risk level
eye_issue_risk	varchar (10)	Eye issue risk level
wound_risk	varchar (10)	Wound risk level
parasite_risk	varchar (10)	Parasite risk level
obesity_risk	varchar (10)	Obesity risk level
overall_risk_level	varchar (10)	Overall calculated risk level
notes	text	User symptom details and comments
scan_image	varchar (file)	Scan image file path

8.1 APPENDIX A. TABLE DESIGNS (Continued)

5. VaccinationRecord

Table name: records_vaccinationrecord | Description: Vaccination records | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
pet_id	int (FK)	Foreign key to pets_pet
vaccine_name	varchar (100)	Vaccine name
scheduled_date	date	Date scheduled for dose
administered_date	date	Date vaccine was given
status	varchar (20)	Vaccination status (scheduled, complete)
vet_name	varchar (100)	Administering veterinarian
notes	text	Dose comments and notes

6. MedicalRecord

Table name: records_medicalrecord | Description: Medical visits | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
pet_id	int (FK)	Foreign key to pets_pet
visit_date	date	Clinic visit date
clinic_name	varchar (150)	Clinic name
diagnosis	text	Doctor diagnosis summary
treatment	text	Administered treatment details
prescription	text	Prescribed medications and instructions
follow_up_date	date	Date for follow-up visit

8.1 APPENDIX A. TABLE DESIGNS (Continued)

7. BreedInfo

Table name: advisory_breedinfo | Description: Breed profiles | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
species	varchar (20)	Species (dog, cat)
breed_name	varchar (100)	Breed name (unique)
description	text	Characteristics and profile
typical_weight_min_kg	decimal (5,2)	Typical minimum weight
typical_weight_max_kg	decimal (5,2)	Typical maximum weight
temperament	varchar (255)	Breed traits
grooming_needs	text	Grooming advice

8. DiseaseInfo

Table name: advisory_diseaseinfo | Description: Disease details | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
species	varchar (20)	Species (dog, cat)
name	varchar (150)	Disease name
common_symptoms	text	List of symptoms
prevention	text	Prevention tips
treatment	text	Treatment guidance

8.1 APPENDIX A. TABLE DESIGNS (Continued)

9. VetDoctor

Table name: advisory_vetdoctor | Description: Registered doctors | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
name	varchar (150)	Doctor full name
specialization	varchar (50)	Specialization field (general, surgery)
phone	varchar (20)	Contact phone number
email	varchar (254)	Contact email address
clinic_name	varchar (200)	Clinic affiliation
address	text	Clinic address
photo	varchar (file)	Doctor profile photo path
experience_years	int	Years of clinical experience
is_available	boolean	Availability status flag

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 1)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: User Registration page layout with email input, password rules validation, and submit button.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 2)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: User Login & OTP page layout featuring email-based login, verification OTP token entry box, and session checks.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 3)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: Pet Profile Dashboard showing registered pets, species choices (dog/cat), weights, and age details.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 4)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: AI Breed Identification panel displaying image drop area, preview of pet, model version choice, and prediction submit button.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 5)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: AI Health Scan panel with health image upload, optional symptom checkboxes, and rule risk mapping submit.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 6)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: Medical Records Log detailing vaccination records, medical visit history, prescriptions, and follow-up dates.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 7)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: Pet Service Finder showing clinic listings, registered vet doctor profiles, and booking date/time forms.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 8)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: Missing Pet Report map screen displaying missing pet photo uploads, sighting details, and contact logs.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 9)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: Admin ML Control Panel showing training dataset ZIP uploads, class distribution statistics, and training execution control buttons.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.3 APPENDIX C. SOURCE CODE

8.3 APPENDIX C. SOURCE CODE (Page 1 of 78)

```
# === FILE: manage.py ===
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import os
import sys

def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'petcare.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()
```

8.3 APPENDIX C. SOURCE CODE (Page 2 of 78)

```
# === FILE: petcare/settings.py ===
"""
Django settings for petcare project.

Generated by 'django-admin startproject' using Django 5.2.11.

For more information on this file, see
https://docs.djangoproject.com/en/5.2/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/5.2/ref/settings/
"""

import os
from pathlib import Path
from dotenv import load_dotenv

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# Load project environment variables. Local .env values should win in
development.
```

8.3 APPENDIX C. SOURCE CODE (Page 3 of 78)

```
load_dotenv(BASE_DIR / '.env', override=True)

# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = os.getenv('SECRET_KEY', 'fallback-insecure-key-for-dev')

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = os.getenv('DEBUG', 'False').lower() in ('true', '1', 'yes')

ALLOWED_HOSTS = [h.strip() for h in os.getenv('ALLOWED_HOSTS',
'').split(',') if h.strip()]

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
```

8.3 APPENDIX C. SOURCE CODE (Page 4 of 78)

```
# Project apps
'accounts',
'pets',
'records',
'analytics',
'advisory',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'petcare.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
```

8.3 APPENDIX C. SOURCE CODE (Page 5 of 78)

```
'DIRS': [BASE_DIR / 'templates'],
'APP_DIRS': True,
'OPTIONS': {
    'context_processors': [
        'django.template.context_processors.request',
        'django.contrib.auth.context_processors.auth',
        'django.contrib.messages.context_processors.messages',
        'records.context_processors.reminder_notifications',
        'pets.context_processors.user_notifications',
    ],
},
],

WSGI_APPLICATION = 'petcare.wsgi.application'

# Database
# https://docs.djangoproject.com/en/5.2/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
```


8.3 APPENDIX C. SOURCE CODE (Page 6 of 78)

```
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

# Password validation
# https://docs.djangoproject.com/en/5.2/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
        'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME':
        'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME':
        'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME':
        'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
```

8.3 APPENDIX C. SOURCE CODE (Page 7 of 78)

```
# Internationalization
# https://docs.djangoproject.com/en/5.2/topics/i18n/

LANGUAGE_CODE = 'en-us'

TIME_ZONE = os.getenv('TIME_ZONE', 'Asia/Kolkata')

USE_I18N = True

USE_TZ = True


# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.2/howto/static-files/

STATIC_URL = 'static/'
STATICFILES_DIRS = [BASE_DIR / 'static']

MEDIA_URL = 'media/'
MEDIA_ROOT = BASE_DIR / 'media'


# Development password-reset emails are printed in the console.
```

8.3 APPENDIX C. SOURCE CODE (Page 8 of 78)

```
EMAIL_BACKEND = os.getenv(
    'EMAIL_BACKEND',
    'django.core.mail.backends.console.EmailBackend',
)
DEFAULT_FROM_EMAIL = os.getenv('DEFAULT_FROM_EMAIL', 'PetCare AI <no-reply@petcare.local>')
EMAIL_HOST = os.getenv('EMAIL_HOST', 'localhost')
EMAIL_PORT = int(os.getenv('EMAIL_PORT', '25'))
EMAIL_HOST_USER = os.getenv('EMAIL_HOST_USER', '')
EMAIL_HOST_PASSWORD = os.getenv('EMAIL_HOST_PASSWORD', '')
EMAIL_USE_TLS = os.getenv('EMAIL_USE_TLS', 'False').lower() in ('true', '1', 'yes')
EMAIL_USE_SSL = os.getenv('EMAIL_USE_SSL', 'False').lower() in ('true', '1', 'yes')

SITE_URL = os.getenv('SITE_URL', 'http://127.0.0.1:8000').rstrip('/')

# Firebase Cloud Messaging settings for browser push notifications.
... [code truncated for report length] ...

# === FILE: petcare/urls.py ===
from django.contrib import admin
from django.urls import include, path
from django.views.generic import TemplateView
from advisory.views import contact
from django.conf import settings
```

8.3 APPENDIX C. SOURCE CODE (Page 9 of 78)

```
from django.conf.urls.static import static
from records.views import firebase_messaging_sw
from .views import home, reviews

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/", include(("accounts.urls", "accounts"),
namespace="accounts")),
    path("advisory/", include("advisory.urls", namespace="advisory")),
    path("pets/", include("pets.urls", namespace="pets")),
    path("records/", include("records.urls", namespace="records")),
    path("firebase-messaging-sw.js", firebase_messaging_sw,
name="firebase_messaging_sw"),
    path(
        "",
        home,
        name="home",
    ),
    path(
        "reviews/",
        reviews,
        name="reviews",
    ),
    path(
```

8.3 APPENDIX C. SOURCE CODE (Page 10 of 78)

```
        "about/",
        TemplateView.as_view(template_name="about.html"),
        name="about",
    ),
    path(
        "contact/",
        contact,
        name="contact",
    ),
    path(
        "terms/",
        TemplateView.as_view(template_name="terms.html"),
        name="terms",
    ),
    path(
        "privacy/",
        TemplateView.as_view(template_name="privacy.html"),
        name="privacy",
    ),
    path("dashboard/", include("analytics.urls",
namespace="analytics"))),
]

if settings.DEBUG:
```

8.3 APPENDIX C. SOURCE CODE (Page 11 of 78)

```
urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)

# === FILE: pets/models.py ===
from django.db import models
from django.contrib.auth import get_user_model

User = get_user_model()

class Pet(models.Model):
    SPECIES_CHOICES = [
        ("dog", "Dog"),
        ("cat", "Cat"),
    ]

    owner = models.ForeignKey(User, on_delete=models.CASCADE,
related_name="pets")
    name = models.CharField(max_length=100)
    species = models.CharField(max_length=20, choices=SPECIES_CHOICES)
    breed = models.CharField(max_length=100, blank=True)
    date_of_birth = models.DateField(null=True, blank=True)
    weight_kg = models.DecimalField(max_digits=5, decimal_places=2,
null=True, blank=True)
```

8.3 APPENDIX C. SOURCE CODE (Page 12 of 78)

```
        image = models.ImageField(upload_to="pet_images/", null=True,
blank=True)
        created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self) -> str:
        return f"{self.name} ({self.species})"

class BreedPrediction(models.Model):
    pet = models.ForeignKey(Pet, on_delete=models.CASCADE,
related_name="breed_predictions")
    predicted_breed = models.CharField(max_length=100)
    species = models.CharField(max_length=20)
    confidence = models.DecimalField(max_digits=5, decimal_places=2,
help_text="Confidence percentage (0-100).")
    model_version = models.CharField(max_length=50, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]

    def __str__(self) -> str:
        return f"{self.pet} -> {self.predicted_breed}
({self.confidence}%)"

class HealthAssessment(models.Model):
```

8.3 APPENDIX C. SOURCE CODE (Page 13 of 78)

```
RISK_LEVEL_CHOICES = [
    ("low", "Low"),
    ("medium", "Medium"),
    ("high", "High"),
]

pet = models.ForeignKey(Pet, on_delete=models.CASCADE,
related_name="health_assessments")
assessment_date = models.DateTimeField(auto_now_add=True)

skin_infection_risk = models.CharField(max_length=10,
choices=RISK_LEVEL_CHOICES, default="low")
fur_loss_risk = models.CharField(max_length=10,
choices=RISK_LEVEL_CHOICES, default="low")
eye_issue_risk = models.CharField(max_length=10,
choices=RISK_LEVEL_CHOICES, default="low")
wound_risk = models.CharField(max_length=10,
choices=RISK_LEVEL_CHOICES, default="low")
parasite_risk = models.CharField(max_length=10,
choices=RISK_LEVEL_CHOICES, default="low")
obesity_risk = models.CharField(max_length=10,
choices=RISK_LEVEL_CHOICES, default="low")

overall_risk_level = models.CharField(max_length=10,
choices=RISK_LEVEL_CHOICES, default="low")
notes = models.TextField(blank=True)
scan_image = models.ImageField(upload_to="health_scans/", null=True,
blank=True)
symptom_details = models.TextField(blank=True, help_text="Optional
symptom details provided by user.")
care_recommendations = models.TextField(blank=True, help_text="AI
generated care recommendations.")

class Meta:
```


8.3 APPENDIX C. SOURCE CODE (Page 14 of 78)

```
ordering = ["-assessment_date"]

def __str__(self) -> str:
    return f"Health assessment for {self.pet} on {self.assessment_date:%Y-%m-%d}"

class MissingPet(models.Model):
    owner = models.ForeignKey(User, on_delete=models.CASCADE,
related_name="missing_pets")
    pet = models.ForeignKey(Pet, on_delete=models.SET_NULL, null=True,
blank=True, related_name="missing_reports")
    pet_name = models.CharField(max_length=100)
    species = models.CharField(max_length=20,
choices=Pet.SPECIES_CHOICES)
    breed = models.CharField(max_length=100, blank=True)
    description = models.TextField()
    last_seen_location = models.CharField(max_length=255)
    last_seen_lat = models.FloatField(null=True, blank=True)
    last_seen_lng = models.FloatField(null=True, blank=True)
    last_seen_date = models.DateTimeField()
    photo = models.ImageField(upload_to="missing_pets/")
    contact_phone = models.CharField(max_length=20)
    is_found = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
```

8.3 APPENDIX C. SOURCE CODE (Page 15 of 78)

```
        ordering = ["-created_at"]

    def __str__(self):
        return f"MISSING: {self.pet_name} ({self.last_seen_location})"

class PetSighting(models.Model):
    missing_pet = models.ForeignKey(MissingPet,
on_delete=models.CASCADE, related_name="sightings")
    sighting_location = models.CharField(max_length=255)
    sighting_date = models.DateTimeField()
    description = models.TextField()
    photo = models.ImageField(upload_to="sightings/", null=True,
blank=True)
    contact_info = models.CharField(max_length=255, help_text="Contact
of the person who saw the pet")
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]

    def __str__(self):
        return f"Sighting for {self.missing_pet.pet_name} at
{self.sighting_location}"

class MissingPetNotification(models.Model):
```

8.3 APPENDIX C. SOURCE CODE (Page 16 of 78)

```
EVENT_CHOICES = [
    ("missing_reported", "Missing pet reported"),
    ("sighting_reported", "Sighting reported"),
    ("marked_found", "Pet marked found"),
    ("reopened_missing", "Missing report reopened"),
]

user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name="missing_pet_notifications")
missing_pet = models.ForeignKey(MissingPet,
on_delete=models.CASCADE, related_name="notifications")
... [code truncated for report length] ...

# === FILE: pets/urls.py ===
from django.urls import path

from . import views

app_name = "pets"

urlpatterns = [
    path("", views.pet_list_view, name="pet_list"),
    path("add/", views.pet_add_view, name="pet_add"),
    path("<int:pet_id>/", views.pet_detail_view, name="pet_detail"),
```

8.3 APPENDIX C. SOURCE CODE (Page 17 of 78)

```
        path("<int:pet_id>/edit/", views.pet_edit_view, name="pet_edit"),
            path("<int:pet_id>/identify/", views.breed_identify_view,
name="breed_identify"),
            path("<int:pet_id>/predict/", views.run_breed_prediction_view,
name="run_breed_prediction"),
            path("<int:pet_id>/health-scan/", views.health_scan_view,
name="health_scan"),
            path("<int:pet_id>/run-health-scan/",
views.run_health_prediction_view, name="run_health_prediction"),
            path("notifications/", views.notifications_view,
name="notifications"),

    # Missing Pets
        path("missing/", views.missing_pets_list_view,
name="missing_pets_list"),
        path("missing/report/", views.report_missing_view,
name="report_missing"),
        path("missing/<int:report_id>/", views.missing_pet_detail_view,
name="missing_pet_detail"),
        path("missing/<int:report_id>/edit/", views.edit_missing_pet_view,
name="edit_missing_pet"),
        path("missing/<int:report_id>/toggle-found/",
views.toggle_missing_pet_found_view, name="toggle_missing_pet_found"),
    ]

# === FILE: pets/forms.py ===
from django import forms

from .models import Pet, MissingPet, PetSighting

class PetForm(forms.ModelForm):
```

8.3 APPENDIX C. SOURCE CODE (Page 18 of 78)

```
class Meta:
    model = Pet
    fields = [
        "name",
        "species",
        "breed",
        "date_of_birth",
        "weight_kg",
        "image",
    ]
    widgets = {
        "name": forms.TextInput(attrs={"class": "form-control"}),
        "species": forms.Select(attrs={"class": "form-control"}),
        "breed": forms.TextInput(
            attrs={
                "class": "form-control",
                "placeholder": "Optional - AI will fill this after
prediction",
            }
        ),
        "date_of_birth": forms.DateInput(
            attrs={"type": "date", "class": "form-control"}
        ),
        "weight_kg": forms.NumberInput(attrs={"class": "form-
control"}),
```

8.3 APPENDIX C. SOURCE CODE (Page 19 of 78)

```
        "image": forms.FileInput(attrs={"class": "form-control"}),
    }

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        # Make it very clear that breed is optional
        if "breed" in self.fields:
            self.fields["breed"].required = False
            self.fields["breed"].label = "Breed (optional)"

class MissingPetForm(forms.ModelForm):
    class Meta:
        model = MissingPet
        fields = [
            "pet", "pet_name", "species", "breed",
            "last_seen_location", "last_seen_lat", "last_seen_lng",
            "last_seen_date",
            "photo", "contact_phone", "description"
        ]
        widgets = {
            "pet_name": forms.TextInput(attrs={"class": "form-control",
            "placeholder": "Pet's name"}),
            "species": forms.Select(attrs={"class": "form-control"}),
            "breed": forms.TextInput(attrs={"class": "form-control",
            "placeholder": "Breed (if known)"}),
```

8.3 APPENDIX C. SOURCE CODE (Page 20 of 78)

```
        "last_seen_location": forms.TextInput(attrs={"class": "form-control", "placeholder": "e.g. City Park, near North Gate"}),
        "last_seen_lat": forms.HiddenInput(),
        "last_seen_lng": forms.HiddenInput(),
        "last_seen_date": forms.DateTimeInput(attrs={"class": "form-control", "type": "datetime-local"}),
        "photo": forms.FileInput(attrs={"class": "form-control"}),
        "contact_phone": forms.TextInput(attrs={"class": "form-control", "placeholder": "Your phone number"}),
        "description": forms.Textarea(attrs={"class": "form-control", "rows": 4, "placeholder": "Any distinctive marks, collar color, etc."}),
        "pet": forms.Select(attrs={"class": "form-control"}),
    }

    def __init__(self, *args, **kwargs):
        user = kwargs.pop("user", None)
        super().__init__(*args, **kwargs)
        if user:
            self.fields["pet"].queryset = Pet.objects.filter(owner=user)
            self.fields["pet"].required = False
            self.fields["pet"].label = "Select from your pets (optional)"

class PetSightingForm(forms.ModelForm):
    class Meta:
        model = PetSighting
        fields = ["sighting_location", "sighting_date", "photo", "contact_info", "description"]
```

8.3 APPENDIX C. SOURCE CODE (Page 21 of 78)

```
        widgets = {
            "sighting_location": forms.TextInput(attrs={"class": "form-control", "placeholder": "Where did you see the pet?"}),
            "sighting_date": forms.DateTimeInput(attrs={"class": "form-control", "type": "datetime-local"}),
            "photo": forms.FileInput(attrs={"class": "form-control"}),
            "contact_info": forms.TextInput(attrs={"class": "form-control", "placeholder": "Your phone or email"}),
            "description": forms.Textarea(attrs={"class": "form-control", "rows": 3, "placeholder": "Behavior, direction of travel, etc."}),
        }

# === FILE: pets/ml_predictor.py ===
"""
ML Predictor - Pet Breed & Health Identification
=====
Loads the trained EfficientNetB0 models and provides
predict_breed(image_path) and predict_health(image_path)
for use in Django views.
"""

import json
import os

import numpy as np
from PIL import Image
```


8.3 APPENDIX C. SOURCE CODE (Page 22 of 78)

```
_breed_model = None
_breed_labels = None
_health_model = None
_health_labels = None

IMG_SIZE     = 224
ML_DIR       =
os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
"ml")
BREED_MODEL_PATH = os.path.join(ML_DIR, "pet_breed_model.keras")
BREED_LABELS_PATH = os.path.join(ML_DIR, "breed_labels.json")
HEALTH_MODEL_PATH = os.path.join(ML_DIR, "pet_health_model.keras")
HEALTH_LABELS_PATH = os.path.join(ML_DIR, "health_labels.json")

def _load_breed_model():
    global _breed_model, _breed_labels
    if _breed_model is None:
        import tensorflow as tf
        _breed_model = tf.keras.models.load_model(BREED_MODEL_PATH,
compile=False)
    if _breed_labels is None:
        with open(BREED_LABELS_PATH) as f:
            _breed_labels = json.load(f)
    return _breed_model, _breed_labels
```

8.3 APPENDIX C. SOURCE CODE (Page 23 of 78)

```
def _load_health_model():
    global _health_model, _health_labels
    if _health_model is None:
        import tensorflow as tf
        _health_model = tf.keras.models.load_model(HEALTH_MODEL_PATH,
compile=False)
    if _health_labels is None:
        with open(HEALTH_LABELS_PATH) as f:
            _health_labels = json.load(f)
    return _health_model, _health_labels

def _preprocess_image(image_path: str) -> np.ndarray:
    # EfficientNet expects raw [0,255] pixels – no manual scaling needed
    img = Image.open(image_path).convert("RGB")
    img = img.resize((IMG_SIZE, IMG_SIZE))
    img_array = np.array(img, dtype=np.float32)
    return np.expand_dims(img_array, axis=0)

def predict_breed(image_path: str) -> dict:
    """Returns dict with top predicted breed details."""
    model, labels = _load_breed_model()
```

8.3 APPENDIX C. SOURCE CODE (Page 24 of 78)

```
img_array = _preprocess_image(image_path)
predictions = model.predict(img_array, verbose=0)[0]

best_idx = int(predictions.argsort()[-1])
best_info = labels[str(best_idx)]
best_conf = float(predictions[best_idx])
top_predictions = []
for idx in predictions.argsort()[-5:][::-1]:
    info = labels[str(int(idx))]
    top_predictions.append(
        {
            "breed": info["breed"],
            "species": info["species"],
            "confidence": round(float(predictions[idx]) * 100, 2),
        }
    )

return {
    "breed": best_info["breed"],
    "species": best_info["species"],
    "confidence": round(best_conf * 100, 2),
    "low_confidence": best_conf < 0.80,
    "top_predictions": top_predictions,
```

8.3 APPENDIX C. SOURCE CODE (Page 25 of 78)

```
    }

def predict_health(image_path: str) -> dict:
    """Returns dict with predicted health conditions."""
    model, labels = _load_health_model()
    img_array = _preprocess_image(image_path)
    predictions = model.predict(img_array, verbose=0)[0]

    best_idx = int(predictions.argsort()[-1])
    best_info = labels[str(best_idx)]
    best_conf = float(predictions[best_idx])

    return {
        "disease": best_info["disease"],
        "description": best_info["description"],
        "confidence": round(best_conf * 100, 2),
        "low_confidence": best_conf < 0.50,
    }

# === FILE: pets/views.py ===
import random
```

8.3 APPENDIX C. SOURCE CODE (Page 26 of 78)

```
import time
from django.db.models import Q
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.shortcuts import get_object_or_404, redirect, render
from django.views.decorators.http import require_POST

from advisory.models import ServiceBooking
from records.models import MedicalRecord, Reminder, VaccinationRecord

from .forms import PetForm, MissingPetForm, PetSightingForm
from .models import BreedPrediction, HealthAssessment, Pet, MissingPet,
MissingPetNotification, PetSighting
from .notifications import (
    notify_found_status_changed,
    notify_missing_report_created,
    notify_sighting_reported,
)

#      —      Breed-specific      health      risk      data      (demo)
# Maps breed names to typical health risks. Unknown breeds get a
# default.
BREED_HEALTH_RISKS = {
    "Golden Retriever": {"skin_infection_risk": "medium",
    "fur_loss_risk": "medium", "obesity_risk": "high"},
```

8.3 APPENDIX C. SOURCE CODE (Page 27 of 78)

```
        "German Shepherd": {"skin_infection_risk": "medium",
"parasite_risk": "medium"},
        "Labrador Retriever": {"obesity_risk": "high", "eye_issue_risk":
"low"},
        "Poodle": {"eye_issue_risk": "medium", "fur_loss_risk": "low"},
        "Bulldog": {"skin_infection_risk": "high", "wound_risk": "medium",
"obesity_risk": "high"},
        "Beagle": {"obesity_risk": "medium", "parasite_risk": "medium"},
        "Rottweiler": {"obesity_risk": "medium", "wound_risk": "medium"},
        "Yorkshire Terrier": {"fur_loss_risk": "medium", "eye_issue_risk":
"medium"},
        "Boxer": {"skin_infection_risk": "medium", "wound_risk": "medium"},
        "Persian": {"eye_issue_risk": "high", "fur_loss_risk": "medium"},
        "Siamese": {"eye_issue_risk": "medium", "skin_infection_risk":
"low"},
        "Maine Coon": {"obesity_risk": "medium", "fur_loss_risk": "low"},
        "Bengal": {"skin_infection_risk": "low", "parasite_risk": "medium"},
        "Sphynx": {"skin_infection_risk": "high", "eye_issue_risk":
"medium"},
        "British Shorthair": {"obesity_risk": "high"},
        "Ragdoll": {"fur_loss_risk": "medium"},
        "Russian Blue": {"obesity_risk": "medium"},
        "Abyssinian": {"parasite_risk": "medium"},
    }

def _calculate_overall_risk(assessment: HealthAssessment) -> str:
    """Derive overall risk from individual risk fields."""
    risk_values = {
```

8.3 APPENDIX C. SOURCE CODE (Page 28 of 78)

```
        "low": 1,
        "medium": 2,
        "high": 3,
    }
    fields = [
        assessment.skin_infection_risk,
        assessment.fur_loss_risk,
        assessment.eye_issue_risk,
        assessment.wound_risk,
        assessment.parasite_risk,
        assessment.obesity_risk,
    ]
    avg = sum(risk_values.get(f, 1) for f in fields) / len(fields)
    if avg >= 2.5:
        return "high"
    elif avg >= 1.5:
        return "medium"
    return "low"
```

#

—

Views

```
@login_required
def pet_list_view(request):
```

8.3 APPENDIX C. SOURCE CODE (Page 29 of 78)

```
    pets = Pet.objects.filter(owner=request.user)
    return render(request, "pets/pet_list.html", {"pets": pets,
"active_section": "my_pets"})

@login_required
def pet_detail_view(request, pet_id):
    pet = get_object_or_404(Pet, id=pet_id, owner=request.user)
    breed_prediction = pet.breed_predictions.first()
    health_assessments = pet.health_assessments.all()
    health_assessment = health_assessments.first()
    pet_bookings = (
        ServiceBooking.objects.filter(user=request.user, pet=pet)
        .select_related("pet_service", "vet_doctor")
        .order_by("-booking_date", "-booking_time", "-created_at")
    )
    medical_records = MedicalRecord.objects.filter(pet=pet).order_by("-visit_date")
    vaccination_records =
VaccinationRecord.objects.filter(pet=pet).order_by(
    "-scheduled_date", "-administered_date"
)
    open_reminders = Reminder.objects.filter(pet=pet,
is_completed=False).order_by(
    "due_date", "due_time", "title"
)
    context = {
```


8.3 APPENDIX C. SOURCE CODE (Page 30 of 78)

```
        "pet": pet,
        "breed_prediction": breed_prediction,
        "health_assessment": health_assessment,
        "health_assessments": health_assessments[:4],
        "pet_bookings": pet_bookings[:5],
        "medical_records": medical_records[:4],
        "vaccination_records": vaccination_records[:4],
        "open_reminders": open_reminders[:4],
        "pet_bookings_count": pet_bookings.count(),
        "health_assessments_count": health_assessments.count(),
        "medical_records_count": medical_records.count(),
        "open_reminders_count": open_reminders.count(),
        "active_section": "my_pets",
    }
    return render(request, "pets/pet_detail.html", context)
```

```
@login_required
def pet_add_view(request):
    dashboard_mode = request.path.startswith("/dashboard/")
    if request.method == "POST":
        form = PetForm(request.POST, request.FILES)
        if form.is_valid():
```

8.3 APPENDIX C. SOURCE CODE (Page 31 of 78)

```
        pet = form.save(commit=False)
        pet.owner = request.user
        pet.save()
        messages.success(request, f"{pet.name} has been added to
your pets.")
        if dashboard_mode:
            return redirect("analytics:my_pets_section")
            return redirect("pets:pet_detail", pet_id=pet.id)
    else:
        form = PetForm()
        template_name = "pets/pet_form_dashboard.html" if dashboard_mode
else "pets/pet_form.html"
    return render(
        request,
        template_name,
        {"form": form, "active_section": "add_pet", "dashboard_mode":
dashboard_mode},
    )

@login_required
def pet_edit_view(request, pet_id):
    dashboard_mode = request.path.startswith("/dashboard/")
    pet = get_object_or_404(Pet, id=pet_id, owner=request.user)
    if request.method == "POST":
        form = PetForm(request.POST, request.FILES, instance=pet)
```

8.3 APPENDIX C. SOURCE CODE (Page 32 of 78)

```
        if form.is_valid():
            form.save()
            messages.success(request, f"{pet.name}'s profile has been
updated.")
            return redirect("pets:pet_detail", pet_id=pet.id)
        else:
            form = PetForm(instance=pet)
            template_name = "pets/pet_form_dashboard.html" if dashboard_mode
else "pets/pet_form.html"
            return render(
                request,
                template_name,
                {"form": form, "pet": pet, "active_section": "my_pets",
"dashboard_mode": dashboard_mode},
                )

@require_POST
@login_required
def run_breed_prediction_view(request, pet_id):
    pet = get_object_or_404(Pet, id=pet_id, owner=request.user)

    if not pet.image:
        messages.error(request, "Cannot run prediction without a pet
image.")
        return redirect("pets:pet_detail", pet_id=pet.id)
```

8.3 APPENDIX C. SOURCE CODE (Page 33 of 78)

```

# — Real ML Model Inference
started_at = time.perf_counter()
try:
    from .ml_predictor import predict_breed
    breed_result = predict_breed(pet.image.path)
    predicted_breed = breed_result["breed"]
    species = breed_result["species"]
    confidence = breed_result["confidence"]
    low_confidence = breed_result.get("low_confidence", confidence <
80)

    top_predictions = breed_result.get("top_predictions", [])
    model_version = "1.0.0-efficientnetb0"
except Exception as e:
    messages.warning(request, f"AI model error ({e}).")
    predicted_breed = "Unknown"
    species = pet.species
    confidence = 0.0
    low_confidence = True
    top_predictions = []
    model_version = "0.1.0-fallback"

elapsed = time.perf_counter() - started_at
print(
    "[breed-predict] "

```

8.3 APPENDIX C. SOURCE CODE (Page 34 of 78)

```
f"pet_id={pet.id} result={predicted_breed} "
f"confidence={confidence:.1f}% elapsed={elapsed:.2f}s"
)

BreedPrediction.objects.create(
    pet=pet,
    predicted_breed=(
        f"Uncertain ({predicted_breed})"
        if low_confidence and predicted_breed != "Unknown"
        else predicted_breed
    ),
    species=species,
    confidence=confidence,
    model_version=model_version,
)

if predicted_breed == "Unknown":
    messages.warning(request, f"AI could not identify {pet.name}'s
breed from this image.")
elif low_confidence:
    alternatives = ", ".join(
        f"{item['breed']} {item['confidence']:.1f}%"
        for item in top_predictions[:3]
    )
```

8.3 APPENDIX C. SOURCE CODE (Page 35 of 78)

```
        messages.warning(
            request,
            (
                f"AI is not confident for {pet.name}. Closest trained
match: "
                f"{predicted_breed} ({confidence:.1f}%)."
                + (f" Top options: {alternatives}." if alternatives else
""))
            ),
        )
    else:
        if not pet.breed:
            pet.breed = predicted_breed
            pet.save(update_fields=["breed"])
            messages.success(request, f"AI identified {pet.name} as a
{predicted_breed} ({confidence:.1f}% confidence).")
            return redirect("pets:pet_detail", pet_id=pet.id)

@login_required
def breed_identify_view(request, pet_id):
    pet = get_object_or_404(Pet, id=pet_id, owner=request.user)
    breed_prediction = pet.breed_predictions.first()
    return render(
        request,
        "pets/breed_identify.html",
```

8.3 APPENDIX C. SOURCE CODE (Page 36 of 78)

```
        {"pet": pet, "breed_prediction": breed_prediction,
"active_section": "my_pets"},
    )

@login_required
def health_scan_view(request, pet_id):
    pet = get_object_or_404(Pet, id=pet_id, owner=request.user)
    assessment = pet.health_assessments.first()
    return render(
        request,
        "pets/health_scan.html",
        {"pet": pet, "health_assessment": assessment, "active_section":
"my_pets"},
    )

@require_POST
@login_required
def run_health_prediction_view(request, pet_id):
    pet = get_object_or_404(Pet, id=pet_id, owner=request.user)
    ... [code truncated for report length] ...

# === FILE: records/models.py ===
from django.db import models
```

8.3 APPENDIX C. SOURCE CODE (Page 37 of 78)

```
from django.conf import settings
from django.utils import timezone
from pets.models import Pet

class VaccinationRecord(models.Model):
    STATUS_CHOICES = [
        ("scheduled", "Scheduled"),
        ("completed", "Completed"),
        ("missed", "Missed"),
    ]

    pet = models.ForeignKey(Pet, on_delete=models.CASCADE,
related_name="vaccinations")
    vaccine_name = models.CharField(max_length=100)
    scheduled_date = models.DateField(null=True, blank=True)
    administered_date = models.DateField(null=True, blank=True)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES,
default="scheduled")
    vet_name = models.CharField(max_length=100, blank=True)
    notes = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-scheduled_date", "-administered_date"]
```


8.3 APPENDIX C. SOURCE CODE (Page 38 of 78)

```
def __str__(self) -> str:
    return f"{self.vaccine_name} for {self.pet}"

class MedicalRecord(models.Model):
    pet = models.ForeignKey(Pet, on_delete=models.CASCADE,
related_name="medical_records")
    visit_date = models.DateField()
    clinic_name = models.CharField(max_length=150, blank=True)
    diagnosis = models.TextField()
    treatment = models.TextField(blank=True)
    prescription = models.TextField(blank=True)
    follow_up_date = models.DateField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-visit_date"]

    def __str__(self) -> str:
        return f"Medical record for {self.pet} on {self.visit_date:%Y-%m-%d}"

class Reminder(models.Model):
```

8.3 APPENDIX C. SOURCE CODE (Page 39 of 78)

```
REMINDER_TYPE_CHOICES = [
    ("vaccination", "Vaccination"),
    ("grooming", "Grooming"),
    ("training", "Training"),
    ("medical", "Medical visit"),
    ("follow_up", "Follow-up"),
    ("custom", "Custom"),
]

pet = models.ForeignKey(Pet, on_delete=models.CASCADE,
related_name="reminders")
title = models.CharField(max_length=160)
reminder_type = models.CharField(max_length=30,
choices=REMINDER_TYPE_CHOICES, default="custom")
due_date = models.DateField()
due_time = models.TimeField(null=True, blank=True)
notes = models.TextField(blank=True)
is_completed = models.BooleanField(default=False)
email_sent_at = models.DateTimeField(null=True, blank=True)
push_sent_at = models.DateTimeField(null=True, blank=True)
created_at = models.DateTimeField(auto_now_add=True)

class Meta:
    ordering = ["is_completed", "due_date", "due_time", "title"]
```

8.3 APPENDIX C. SOURCE CODE (Page 40 of 78)

```
@property
def is_overdue(self) -> bool:
    if self.is_completed:
        return False
    today = timezone.localdate()
    if self.due_date < today:
        return True
    if self.due_date == today and self.due_time:
        return self.due_time < timezone.localtime().time()
    return False

def __str__(self) -> str:
    due_label = self.due_date.strftime("%Y-%m-%d")
    if self.due_time:
        due_label += f" {self.due_time:%H:%M}"
    return f"{self.title} for {self.pet} on {due_label}"

class PushNotificationToken(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE, related_name="push_tokens")
    token = models.TextField(unique=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
```

8.3 APPENDIX C. SOURCE CODE (Page 41 of 78)

```
updated_at = models.DateTimeField(auto_now=True)

class Meta:
    ordering = ["-updated_at"]

def __str__(self) -> str:
    return f"Push token for {self.user.email or self.user_id}"

# === FILE: records/urls.py ===
from django.urls import path

from . import views

app_name = "records"

urlpatterns = [
    path("", views.records_home, name="records_home"),
        path("vaccinations/add/", views.add_vaccination_record,
name="add_vaccination"),
    path("medical/add/", views.add_medical_record, name="add_medical"),
    path("reminders/", views.reminders_view, name="reminders"),
    path("push-token/", views.save_push_token, name="save_push_token"),
```

8.3 APPENDIX C. SOURCE CODE (Page 42 of 78)

```
        path("reminders/<int:reminder_id>/complete/",
views.complete_reminder, name="complete_reminder"),
        path("reminders/<int:reminder_id>/delete/", views.delete_reminder,
name="delete_reminder"),
    ]

# === FILE: records/views.py ===
import json

from django.conf import settings
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.http import HttpResponse, JsonResponse
from django.shortcuts import get_object_or_404, redirect, render
from django.utils import timezone
from django.views.decorators.http import require_POST

from .forms import MedicalRecordForm, ReminderForm,
VaccinationRecordForm
from .models import MedicalRecord, PushNotificationToken, Reminder,
VaccinationRecord

def _user_pet_ids(user):
    return user.pets.values_list("id", flat=True)
```

8.3 APPENDIX C. SOURCE CODE (Page 43 of 78)

```
def _create_reminder_from_vaccination(record):
    if record.status == "completed" or not record.scheduled_date:
        return
    Reminder.objects.get_or_create(
        pet=record.pet,
        reminder_type="vaccination",
        due_date=record.scheduled_date,
        title=f"Vaccination: {record.vaccine_name}",
        defaults={"notes": record.notes},
    )

def _create_reminder_from_follow_up(record):
    if not record.follow_up_date:
        return
    Reminder.objects.get_or_create(
        pet=record.pet,
        reminder_type="follow_up",
        due_date=record.follow_up_date,
        title=f"Follow-up visit: {record.clinic_name or
record.pet.name}",
        defaults={"notes": record.treatment or record.diagnosis},
    )
```

8.3 APPENDIX C. SOURCE CODE (Page 44 of 78)

```
@login_required
def records_home(request):
    pet_ids = _user_pet_ids(request.user)
    today = timezone.localdate()
    vaccinations = (
        VaccinationRecord.objects.filter(pet__in=pet_ids)
        .select_related("pet")
        .order_by("-scheduled_date", "-administered_date")
    )
    medical_records = (
        MedicalRecord.objects.filter(pet__in=pet_ids)
        .select_related("pet")
        .order_by("-visit_date")
    )
    reminders = (
        Reminder.objects.filter(pet__in=pet_ids)
        .select_related("pet")
        .order_by("is_completed", "due_date")
    )

    context = {
```

8.3 APPENDIX C. SOURCE CODE (Page 45 of 78)

```
        "vaccinations": vaccinations,
        "medical_records": medical_records,
        "reminders": reminders[:8],
            "overdue_count": reminders.filter(is_completed=False,
due_date__lt=today).count(),
            "upcoming_count": reminders.filter(is_completed=False,
due_date__gte=today).count(),
        }
    return render(request, "records/records_home.html", context)

@login_required
def add_vaccination_record(request):
    if request.method == "POST":
        form = VaccinationRecordForm(request.POST, user=request.user)
        if form.is_valid():
            record = form.save()
            _create_reminder_from_vaccination(record)
            messages.success(request, "Vaccination record saved.")
            return redirect("analytics:records_section")
    else:
        form = VaccinationRecordForm(user=request.user)
    return render(
        request,
        "records/record_form.html",
```


8.3 APPENDIX C. SOURCE CODE (Page 46 of 78)

```
        {
            "form": form,
            "title": "Add Vaccination Record",
            "submit_label": "Save Vaccination",
            "active_section": "records",
        },
    ),

@login_required
def add_medical_record(request):
    if request.method == "POST":
        form = MedicalRecordForm(request.POST, user=request.user)
        if form.is_valid():
            record = form.save()
            _create_reminder_from_follow_up(record)
            messages.success(request, "Medical record saved.")
            return redirect("analytics:records_section")
    else:
        form = MedicalRecordForm(user=request.user)
    return render(
        request,
        "records/record_form.html",
```

8.3 APPENDIX C. SOURCE CODE (Page 47 of 78)

```
        {
            "form": form,
            "title": "Add Medical Record",
            "submit_label": "Save Medical Record",
            "active_section": "records",
        },
    )

@login_required
def reminders_view(request):
    pet_ids = _user_pet_ids(request.user)
    if request.method == "POST":
        form = ReminderForm(request.POST, user=request.user)
        if form.is_valid():
            form.save()
            messages.success(request, "Reminder created.")
            return redirect("records:reminders")
    else:
        form = ReminderForm(user=request.user)

    reminders = (
        Reminder.objects.filter(pet__in=pet_ids)
```

8.3 APPENDIX C. SOURCE CODE (Page 48 of 78)

```
        .select_related("pet")
        .order_by("is_completed", "due_date")
    )
    return render(
        request,
        "records/reminders.html",
        {"form": form, "reminders": reminders, "active_section":
"reminders"},
    )

@require_POST
@login_required
def complete_reminder(request, reminder_id):
    reminder = get_object_or_404(Reminder, id=reminder_id,
pet__owner=request.user)
    reminder.is_completed = True
    reminder.save(update_fields=["is_completed"])
    messages.success(request, "Reminder marked as completed.")
    return redirect("records:reminders")
... [code truncated for report length] ...

# === FILE: advisory/models.py ===
from django.db import models
from django.conf import settings
```

8.3 APPENDIX C. SOURCE CODE (Page 49 of 78)

```
class BreedInfo(models.Model):
    SPECIES_CHOICES = [
        ("dog", "Dog"),
        ("cat", "Cat"),
    ]

    species = models.CharField(max_length=20, choices=SPECIES_CHOICES)
    breed_name = models.CharField(max_length=100)
    description = models.TextField(blank=True)
    typical_weight_min_kg = models.DecimalField(max_digits=5,
decimal_places=2, null=True, blank=True)
    typical_weight_max_kg = models.DecimalField(max_digits=5,
decimal_places=2, null=True, blank=True)
    temperament = models.CharField(max_length=255, blank=True)
    grooming_needs = models.TextField(blank=True)
    exercise_needs = models.TextField(blank=True)

    class Meta:
        unique_together = ("species", "breed_name")
        verbose_name = "Breed information"
        verbose_name_plural = "Breed information"

    def __str__(self) -> str:
```

8.3 APPENDIX C. SOURCE CODE (Page 50 of 78)

```
        return f"{self.breed_name} ({self.species})"

class DiseaseInfo(models.Model):
    SPECIES_CHOICES = [
        ("dog", "Dog"),
        ("cat", "Cat"),
    ]

    species = models.CharField(max_length=20, choices=SPECIES_CHOICES)
    name = models.CharField(max_length=150)
    common_symptoms = models.TextField()
    prevention = models.TextField(blank=True)
    treatment = models.TextField(blank=True)

    class Meta:
        unique_together = ("species", "name")

    def __str__(self) -> str:
        return f"{self.name} ({self.species})"

class VaccinationScheduleTemplate(models.Model):
```

8.3 APPENDIX C. SOURCE CODE (Page 51 of 78)

```
SPECIES_CHOICES = [
    ("dog", "Dog"),
    ("cat", "Cat"),
]

species = models.CharField(max_length=20, choices=SPECIES_CHOICES)
vaccine_name = models.CharField(max_length=100)
recommended_age_weeks =
models.PositiveIntegerField(help_text="Recommended age in weeks for
first dose.")
repeat_interval_weeks = models.PositiveIntegerField(null=True,
blank=True, help_text="Interval for booster doses, if applicable.")
notes = models.TextField(blank=True)

class Meta:
    unique_together = ("species", "vaccine_name",
"recommended_age_weeks")
    verbose_name = "Vaccination schedule template"

def __str__(self) -> str:
    return f"{self.vaccine_name} ({self.species})"

class CareAdvisory(models.Model):
    SPECIES_CHOICES = [
        ("dog", "Dog"),
        ("cat", "Cat"),
```

8.3 APPENDIX C. SOURCE CODE (Page 52 of 78)

```
        ("both", "Dog & Cat"),
    ]
    CATEGORY_CHOICES = [
        ("vaccination", "Vaccination schedule"),
        ("diet", "Diet & nutrition"),
        ("grooming", "Grooming practices"),
        ("hygiene", "Hygiene care"),
        ("parasite", "Parasite prevention"),
        ("treatment", "Treatment suggestions"),
    ]

    species = models.CharField(max_length=20, choices=SPECIES_CHOICES,
default="both")
    category = models.CharField(max_length=30, choices=CATEGORY_CHOICES)
    title = models.CharField(max_length=160)
    short_summary = models.CharField(max_length=255, blank=True)
    recommendation = models.TextField()
    when_to_apply = models.CharField(max_length=255, blank=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ["category", "species", "title"]
```

8.3 APPENDIX C. SOURCE CODE (Page 53 of 78)

```
        verbose_name = "Care advisory"
        verbose_name_plural = "Care advisories"

    def __str__(self) -> str:
        return f"{self.title} ({self.get_category_display()})"

class MedicineRecommendation(models.Model):
    SPECIES_CHOICES = CareAdvisory.SPECIES_CHOICES

    species = models.CharField(max_length=20, choices=SPECIES_CHOICES,
default="both")
    condition_name = models.CharField(max_length=150)
    medicine_name = models.CharField(max_length=150)
    dosage_guidance = models.TextField(
        help_text="Use vet-safe guidance only. Avoid exact dosage unless
prescribed."
    )
    administration_notes = models.TextField(blank=True)
    warnings = models.TextField(blank=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["species", "condition_name", "medicine_name"]
```


8.3 APPENDIX C. SOURCE CODE (Page 54 of 78)

```
        unique_together = ("species", "condition_name", "medicine_name")

def __str__(self) -> str:
    return f"{self.medicine_name} for {self.condition_name}"

class PetService(models.Model):
    SERVICE_TYPE_CHOICES = [
        ("vet", "Veterinary clinic"),
        ("groomer", "Groomer"),
        ("trainer", "Trainer"),
        ("boarding", "Boarding"),
        ("pharmacy", "Pet pharmacy"),
    ]

    name = models.CharField(max_length=180)
    service_type = models.CharField(max_length=30,
choices=SERVICE_TYPE_CHOICES)
    city = models.CharField(max_length=100, blank=True)
    address = models.TextField(blank=True)
    phone = models.CharField(max_length=30, blank=True)
    email = models.EmailField(blank=True)
    website = models.URLField(blank=True)
    opening_hours = models.CharField(max_length=160, blank=True)
```

8.3 APPENDIX C. SOURCE CODE (Page 55 of 78)

```
notes = models.TextField(blank=True)
is_active = models.BooleanField(default=True)
created_at = models.DateTimeField(auto_now_add=True)

class Meta:
    ordering = ["city", "service_type", "name"]

def __str__(self) -> str:
    return f"{self.name} ({self.get_service_type_display()})"

... [code truncated for report length] ...

# === FILE: templates/accounts/login.html ===
{% extends "base.html" %}
{% load static %}

{% block content %}
<div class="row justify-content-center py-5">
    <div class="col-lg-10">
        <div class="row g-0 auth-card shadow-lg rounded-4 overflow-
hidden">
            <!-- Illustration / marketing side -->
            <div class="col-lg-6 d-none d-lg-flex flex-column justify-
content-between auth-illustration p-4">
                <div>
```

8.3 APPENDIX C. SOURCE CODE (Page 56 of 78)

```
auth-badge mb-3">
    <span class="badge rounded-pill bg-light text-dark
    <span class="badge-dot"></span>
    Smart care for your pets
</span>
    <h2 class="fw-bold mb-3">Welcome back to PetCare
AI</h2>
    <p class="mb-4 auth-text">
        Enter your email and verify the OTP sent to your
inbox.
    </p>
    <ul class="list-unstyled small mb-0">
        <li class="d-flex align-items-center mb-2">
            <span class="check-icon me-2"></span>
            Personalized health recommendations
        </li>
        <li class="d-flex align-items-center mb-2">
            <span class="check-icon me-2"></span>
            Secure pet records in one place
        </li>
        <li class="d-flex align-items-center">
            <span class="check-icon me-2"></span>
            AI-powered breed identification
        </li>
    </ul>
</div>
```

8.3 APPENDIX C. SOURCE CODE (Page 57 of 78)

```
<div class="text-end mt-4">
    
</div>
</div>

<!-- Form side -->
<div class="col-lg-6 bg-white p-4 p-lg-5">
    <div class="mb-4 text-center text-lg-start">
        <span class="text-muted small text-uppercase fw-
semibold tracking-wide">Login</span>
        <h3 class="fw-bold mt-1 mb-2">Sign in with email</
h3>
        <p class="text-muted small mb-0">Use your registered
email address. No password is needed.</p>
    </div>

    <form method="post" novalidate>
        {% csrf_token %}
        {% if redirect_field_value %}
            <input type="hidden" name="{{
redirect_field_name }}" value="{{ redirect_field_value }}">
        {% endif %}
        {{ form.non_field_errors }}
        {% for field in form %}
            <div class="mb-3">
                <label class="form-label fw-semibold small
text-uppercase" for="{{ field.id_for_label }}">
                    {{ field.label }}
            </div>
        {% endfor %}
    </form>
</div>
```

8.3 APPENDIX C. SOURCE CODE (Page 58 of 78)

```

        </label>
        {{ field }}
        {% if field.errors %}
            <div class="text-danger small mt-1">
                {{ field.errors|striptags }}
            </div>
        {% endif %}
    </div>
{% endfor %}

    <button type="submit" class="btn btn-primary w-100
mt-2">Send login OTP</button>
</form>

    <div class="mt-3 text-center">
        <small class="text-muted">
            Don't have an account?
            <a href="{% url 'accounts:register' %}"
class="auth-link">Create one now</a>
        </small>
    </div>
</div>
</div>
</div>
```

8.3 APPENDIX C. SOURCE CODE (Page 59 of 78)

```
{% endblock %}

# === FILE: templates/accounts/register.html ===
{% extends "base.html" %}
{% load static %}

{% block content %}
<div class="row justify-content-center py-5">
  <div class="col-lg-10">
    <div class="row g-0 auth-card shadow-lg rounded-4 overflow-
hidden">
      <!-- Illustration / marketing side -->
      <div class="col-lg-6 d-none d-lg-flex flex-column justify-
content-between auth-illustration p-4">
        <div>
          <span class="badge rounded-pill bg-light text-dark
auth-badge mb-3">
            <span class="badge-dot"></span>
            Create your pet hub
          </span>
          <h2 class="fw-bold mb-3">Join PetCare AI in
minutes</h2>
          <p class="mb-4 auth-text">
            Register and verify your email to unlock
intelligent health insights, vaccination reminders, and rich profiles
for every pet you love.
          </p>
          <ul class="list-unstyled small mb-0">
```

8.3 APPENDIX C. SOURCE CODE (Page 60 of 78)

```

        <li class="d-flex align-items-center mb-2">
            <span class="check-icon me-2"></span>
            Beautiful dashboards for each pet
        </li>
        <li class="d-flex align-items-center mb-2">
            <span class="check-icon me-2"></span>
            Email OTP verification before account
activation
        </li>
        <li class="d-flex align-items-center">
            <span class="check-icon me-2"></span>
            Built with privacy and security in mind
        </li>
    </ul>
</div>
<div class="text-end mt-4">
    
    </div>
</div>

<!-- Form side -->
<div class="col-lg-6 bg-white p-4 p-lg-5">
    <div class="mb-4 text-center text-lg-start">
        <span class="text-muted small text-uppercase fw-
semibold tracking-wide">Register</span>

```

8.3 APPENDIX C. SOURCE CODE (Page 61 of 78)

```
<h3 class="fw-bold mt-1 mb-2">Create your PetCare AI
account</h3>

    <p class="text-muted small mb-0">Use your email
address. We will send an OTP before login.</p>
</div>

<form method="post" novalidate>
    {% csrf_token %}
    {{ form.non_field_errors }}
    {% for field in form %}
        <div class="mb-3">
            <label class="form-label fw-semibold small
text-uppercase" for="{{ field.id_for_label }}">
                {{ field.label }}
            </label>
            {{ field }}
            {% if field.help_text %}
                <div class="form-text small text-
muted">{{ field.help_text|safe }}</div>
            {% endif %}
            {% for error in field.errors %}
                <div class="text-danger small mt-1">
                    {{ error }}
                </div>
            {% endfor %}
        </div>
    {% endfor %}
</form>
```


8.3 APPENDIX C. SOURCE CODE (Page 62 of 78)

```

        <button type="submit" class="btn btn-primary w-100
mt-2">Create account</button>
    </form>

    <div class="mt-4 text-center">
        <small class="text-muted">
            Already have an account?
            <a href="{% url 'accounts:login' %}"
class="auth-link">Sign in instead</a>
        </small>
        <div class="small text-muted mt-2">
            If your email is already registered, use the
sign-in page to receive a login OTP.
        </div>
    </div>
</div>
</div>
</div>
</div>
{% endblock %}

# === FILE: templates/pets/breed_identify.html ===
{% extends "base.html" %}
```

8.3 APPENDIX C. SOURCE CODE (Page 63 of 78)

```
{% block content %}
<div class="container py-4">
  <!-- Breadcrumb -->
  <nav aria-label="breadcrumb" class="mb-4">
    <ol class="breadcrumb small">
      <li class="breadcrumb-item"><a href="{% url 'pets:pet_list'
%}" class="text-decoration-none" style="color: #6366f1;">My Pets</a></li>

      <li class="breadcrumb-item"><a href="{% url
'pets:pet_detail' pet.id %}" class="text-decoration-none" style="color:
#6366f1;">{{ pet.name }}</a></li>
      <li class="breadcrumb-item active" aria-current="page">Breed
Identification</li>
    </ol>
  </nav>

  <!-- Back Button -->
  <a href="{% url 'pets:pet_detail' pet.id %}" class="btn btn-outline-
secondary btn-sm rounded-pill px-3 mb-3 d-inline-flex align-items-center
gap-2">
    <svg xmlns="http://www.w3.org/2000/svg" width="14" height="14"
fill="currentColor" viewBox="0 0 16 16"><path fill-rule="evenodd" d="M15
8a.5.5 0 0 0-.5-.5H2.70713.147-3.146a.5.5 0 1 0-.708-.708l-4 4a.5.5 0 0
0 0 .708l4 4a.5.5 0 0 0 .708-.708L2.707 8.5H14.5A.5.5 0 0 0 15 8z"/></
svg>
    Back to {{ pet.name }}
  </a>

  <div class="row pt-2">
    <!-- Left Column: Pet Image -->
    <div class="col-md-5">
      <div class="bg-white rounded-4 shadow-sm overflow-hidden
mb-4" style="border: 1px solid rgba(99, 102, 241, 0.25);">
        {% if pet.image %}
          
        {% else %}
          <img alt="Placeholder image for {{ pet.name }}" style="height: 400px;">
        {% endif %}
      </div>
    </div>
  </div>
</div>
{% endblock %}
```

8.3 APPENDIX C. SOURCE CODE (Page 64 of 78)

```

        {% else %}
            <div class="bg-light d-flex align-items-center
justify-content-center w-100" style="height: 400px;">
                <div class="text-center text-muted">
                    <svg xmlns="http://www.w3.org/2000/svg"
width="64" height="64" fill="currentColor" class="mb-2 opacity-25"
viewBox="0 0 16 16"><path d="M6 2a.5.5 0 0 1 .47.33L10
12.036l11.53-4.208A.5.5 0 0 1 12 7.5h3.5a.5.5 0 0 1 0 1h-3.15l-1.88
5.17a.5.5 0 0 1-.94 0L6 3.964 4.47 8.171A.5.5 0 0 1 4 8.5H.5a.5.5 0 0 1
0-1h3.15l1.88-5.17A.5.5 0 0 1 6 2Z"/></svg>
                    <p class="mb-0 small">No Image Uploaded</p>
                </div>
            </div>
        {% endif %}
    </div>

    <!-- Run Identification Button -->
    {% if pet.image %}
        <div class="text-center mt-3 p-3 bg-light rounded-4 border">
            <form action="{% url 'pets:run_breed_prediction' pet.id
%}" method="post">
                {% csrf_token %}
                <p class="small text-muted mb-3 text-start fw-
medium">Upload a clear photo of your pet for best results. The AI model
analyzes visual features like fur, face shape, and body structure.</p>
                <button type="submit" class="btn btn-lg rounded-pill
px-5 py-3 shadow-sm w-100 d-flex align-items-center justify-content-
center gap-2" style="background: #6366f1; color: white; border: none;"
onclick="this.innerHTML='<span class=\''spinner-border spinner-border-
sm\''></span> Identifying Breed...'; this.classList.add('disabled');">
                    <svg xmlns="http://www.w3.org/2000/svg"
width="24" height="24" fill="currentColor" viewBox="0 0 16 16"><path
d="M11.742 10.344a6.5 6.5 0 1 0-1.397
1.398h-.001c.03.04.062.078.098.115l3.85 3.85a1 1 0 0 0
1.415-1.414l-3.85-3.85a1.007 1.007 0 0 0-.115-.115M12 6.5a5.5 5.5 0 1
1-11 0 5.5 5.5 0 0 1 11 0z"/></svg>
                    Run AI Breed Identification
                </button>
                <small class="text-muted d-block mt-3">Powered by
EfficientNet Computer Vision AI.</small>
            </form>
        </div>
    
```

8.3 APPENDIX C. SOURCE CODE (Page 65 of 78)

```

        {% else %}
        <div class="alert alert-warning d-flex align-items-center
gap-2">
            <svg xmlns="http://www.w3.org/2000/svg" width="20"
height="20" fill="currentColor" viewBox="0 0 16 16"><path d="M8 16A8 8 0
1 0 8 0a8 8 0 0 0 0 16zm.93-9.412-1 4.705c-.07.34.029.533.304.533.194 0
.487-.07.686-.246l-.088.416c-.287.346-.92.598-1.465.598-.703
0-1.002-.422-.808-1.319l.738-3.468c.064-.293.006-.399-.287-.471-.451-.081.
2.29-.287zM8 5.5a1 1 0 1 1 0-2 1 1 0 0 1 0 2z"/></svg>
            <small>Please upload a pet image first to run breed
identification.</small>
        </div>
        {% endif %}
    </div>

    <!-- Right Column: Results -->
    <div class="col-md-7 ps-md-4">
        <div class="d-flex align-items-center gap-3 mb-4 mt-4 mt-
md-0">
            <div class="rounded-circle d-flex align-items-center
justify-content-center shadow-sm" style="width: 56px; height: 56px;
background: #6366f1; color: white;">
                <svg xmlns="http://www.w3.org/2000/svg" width="28"
height="28" fill="currentColor" viewBox="0 0 16 16"><path d="M11.742
10.344a6.5 6.5 0 1 0-1.397 1.398h-.001c.03.04.062.078.098.115l3.85
3.85a1 1 0 0 0 1.415-1.414l-3.85-3.85a1.007 1.007 0 0 0-.115-.1zM12
6.5a5.5 5.5 0 1 1-11 0 5.5 5.5 0 0 1 11 0z"/></svg>
            </div>
            <div>
                <h2 class="fw-bold mb-0">Breed Identification</h2>
                <p class="text-muted mb-0">AI-powered breed
detection for {{ pet.name }}</p>
            </div>
        </div>

        <div class="bg-white rounded-4 shadow-sm p-4" style="border-
top: 4px solid #6366f1;">
            {% if breed_prediction %}
                <!-- Prediction Result -->

```

8.3 APPENDIX C. SOURCE CODE (Page 66 of 78)

```

        <div class="row mb-4">
            <div class="col-12">
                <h5 class="fw-bold mb-3 border-bottom pb-2"
style="color: #6366f1;">Prediction Result</h5>
            </div>
            <div class="col-md-7">
                <p class="text-muted small mb-1">AI
Identified Breed</p>
                <h3 class="fw-bold mb-1" style="color:
#6366f1;">{{ breed_prediction.predicted_breed }}</h3>
                <small class="text-muted">Species: {{
breed_prediction.species|title }} . Model: {{
breed_prediction.model_version }}</small>
            </div>
            <div class="col-md-5 text-md-end mt-3 mt-md-0">
                <p class="text-muted small mb-1">Confidence
Level</p>
                {% if breed_prediction.confidence >= 80 %}
                    <span class="badge bg-success rounded-
pill px-4 py-2 fs-5 shadow-sm">{{ breed_prediction.confidence }}%</span>
                {% elif breed_prediction.confidence >= 50 %}
                    <span class="badge bg-warning text-dark
rounded-pill px-4 py-2 fs-5 shadow-sm">{{ breed_prediction.confidence
}}%</span>
                {% else %}
                    <span class="badge bg-danger rounded-
pill px-4 py-2 fs-5 shadow-sm">{{ breed_prediction.confidence }}%</span>
                {% endif %}
            </div>
        </div>

        <!-- Confidence Bar -->
        <div class="mb-4">

```

8.3 APPENDIX C. SOURCE CODE (Page 67 of 78)

```

                                <div class="d-flex justify-content-between
align-items-center mb-2">
                                <small class="fw-medium text-
muted">Detection Confidence</small>
                                <small class="fw-bold" style="color:
#6366f1;">{{ breed_prediction.confidence }}%</small>
                                </div>

                                <div class="progress rounded-pill"
style="height: 14px;">
                                <div class="progress-bar rounded-pill"
role="progressbar" style="width: {{ breed_prediction.confidence }}%;
background: linear-gradient(90deg, #6366f1, #818cf8);" aria-valuenow="{{
breed_prediction.confidence }}" aria-valuemin="0" aria-valuemax="100"></
div>
... [code truncated for report length] ...

# === FILE: templates/pets/health_scan.html ===
{% extends "base.html" %}

{% block content %}
<div class="container py-4">
    <!-- Breadcrumb -->
    <nav aria-label="breadcrumb" class="mb-4">
        <ol class="breadcrumb small">
            <li class="breadcrumb-item"><a href="{% url 'pets:pet_list'
%}" class="text-decoration-none text-success">My Pets</a></li>
            <li class="breadcrumb-item"><a href="{% url
'pets:pet_detail' pet.id %}" class="text-decoration-none text-
success">{{ pet.name }}</a></li>
            <li class="breadcrumb-item active" aria-
current="page">Health Scan</li>
        </ol>
    </nav>

```

8.3 APPENDIX C. SOURCE CODE (Page 68 of 78)

```

        <!-- Back Button -->
        <a href="{% url 'pets:pet_detail' pet.id %}" class="btn btn-outline-
secondary btn-sm rounded-pill px-3 mb-3 d-inline-flex align-items-center
gap-2">
            <svg xmlns="http://www.w3.org/2000/svg" width="14" height="14"
fill="currentColor" viewBox="0 0 16 16"><path fill-rule="evenodd" d="M15
8a.5.5 0 0 0-.5-.5H2.707l3.147-3.146a.5.5 0 1 0-.708-.708l-4 4a.5.5 0 0
0 0 .708l4 4a.5.5 0 0 0 .708-.708L2.707 8.5H14.5a.5.5 0 0 0 1.5 8z"/></
svg>
            Back to {{ pet.name }}
        </a>

    <div class="row pt-2">
        <div class="col-md-5">
            <div class="bg-white rounded-4 shadow-sm overflow-hidden
mb-4 border border-success border-opacity-25">
                {% if health_assessment and health_assessment.scan_image
%}
                    
                    <div class="bg-success text-white text-center py-1
small fw-medium">Last Scanned Image</div>
                {% elif pet.image %}
                    
                {% else %}
                    <div class="bg-light d-flex align-items-center
justify-content-center w-100" style="height: 400px;">
                        <span class="text-muted">No Image Available</
span>
                    </div>
                {% endif %}
            </div>

            {% if pet.image %}

```

8.3 APPENDIX C. SOURCE CODE (Page 69 of 78)

```

        <div class="text-center mt-3 p-3 bg-light rounded-4 border">
            <form action="{% url 'pets:run_health_prediction' pet.id
            %}" method="post" enctype="multipart/form-data">
                {% csrf_token %}
                <p class="small text-muted mb-2 text-start fw-
                medium">Upload a specific photo showing the affected skin/eyes
                (optional). If not provided, the profile picture will be scanned.</p>
                <input type="file" name="scan_image" class="form-
                control mb-3" accept="image/*">
                <textarea name="symptom_details" class="form-control
                mb-3" rows="3" placeholder="Describe any symptoms (e.g., scratching,
                lethargy, loss of appetite) - Optional"></textarea>
                <button type="submit" class="btn btn-success btn-lg
                rounded-pill px-5 py-3 shadow-sm w-100 d-flex align-items-center
                justify-content-center gap-2" onclick="this.innerHTML='<span
                class=\`spinner-border spinner-border-sm\`></span> Scanning Image...';
                this.classList.add('disabled');">
                    <svg xmlns="http://www.w3.org/2000/svg"
                    width="24" height="24" fill="currentColor" viewBox="0 0 16 16"><path
                    d="M8 15A7 7 0 1 1 8 1a7 7 0 0 1 0 14zm0 1A8 8 0 1 0 8 0a8 8 0 0 0 0
                    16z"/><path d="M10.97 4.97a.235.235 0 0 0-.02.022L7.477 9.417 5.384
                    7.323a.75.75 0 0 0-1.06 1.06L6.97 11.03a.75.75 0 0 0 0
                    1.079-.0213.992-4.99a.75.75 0 0 0-1.071-1.05z"/></svg>
                    Run AI Health Scan
                </button>
                <small class="text-muted d-block mt-3">Powered by
                Advanced Computer Vision AI.</small>
            </form>
        </div>
    {% endif %}
</div>

<div class="col-md-7 ps-md-4">
    <div class="d-flex align-items-center gap-3 mb-4 mt-4 mt-
    md-0">
        <div class="bg-success text-white rounded-circle d-flex
        align-items-center justify-content-center shadow-sm" style="width: 56px;
        height: 56px;">
            <svg xmlns="http://www.w3.org/2000/svg" width="28"
            height="28" fill="currentColor" viewBox="0 0 16 16"><path fill-
            rule="evenodd" d="M8 1a.5.5 0 0 1 .5.5v13a.5.5 0 0 1-1 0v-13A.5.5 0 0 1
            8 1z"/><path fill-rule="evenodd" d="M1 8a.5.5 0 0 1 .5-.5h13a.5.5 0 0 1
            0 1h-13A.5.5 0 0 1 1 8z"/></svg>
        </div>
    </div>

```


8.3 APPENDIX C. SOURCE CODE (Page 70 of 78)

```
<h2 class="fw-bold mb-0">Health Diagnostic</h2>
  <p class="text-muted mb-0">Analyzes fur, eyes, and
skin conditions visually.</p>
</div>
</div>

<div class="bg-white rounded-4 shadow-sm p-4 border-top
border-4 border-success">
  {% if health_assessment %}
    <div class="row mb-4">
      <div class="col-12">
        <h5 class="fw-bold text-success mb-3 border-
bottom pb-2">Diagnostic Summary</h5>
      </div>
      <div class="col-md-7">
        <p class="text-muted small mb-1">AI Detected
Condition</p>
        <h4 class="text-dark fw-bold mb-3">{{
health_assessment.notes }}</h4>
      </div>
      <div class="col-md-5 text-md-end">
        <p class="text-muted small mb-1">Overall
System Risk</p>
        {% if health_assessment.overall_risk_level
== "low" %}
          <span class="badge bg-success rounded-
pill px-4 py-2 fs-5 shadow-sm">🟢 LOW</span>
        {% elif health_assessment.overall_risk_level
== "medium" %}
          <span class="badge bg-warning text-dark
rounded-pill px-4 py-2 fs-5 shadow-sm">🟡 MEDIUM</span>
        {% else %}

```

8.3 APPENDIX C. SOURCE CODE (Page 71 of 78)

```

                                <span class="badge bg-danger rounded-
pill px-4 py-2 fs-5 shadow-sm">● HIGH</span>
                                {% endif %}
                                </div>
                                </div>

                                {% if health_assessment.symptom_details %}
                                <div class="bg-light p-3 rounded-3 mb-4">
                                    <span class="text-muted d-block small fw-bold
mb-1">Reported Symptoms</span>
                                <p class="mb-0 text-dark">{{
health_assessment.symptom_details }}</p>
                                </div>
                                {% endif %}

                                <h5 class="fw-bold text-success mb-3 border-bottom
pb-2">Health Risk Indicators</h5>
                                <div class="row g-3">
                                    <div class="col-sm-6">
                                        <div class="border rounded-3 p-3">
                                            <span class="text-muted d-block small
fw-bold">Skin Infection</span>
                                            <span class="fs-5 text-capitalize text-
dark fw-medium">{{ health_assessment.skin_infection_risk }}</span>
                                        </div>
                                    </div>
                                    <div class="col-sm-6">
... [code truncated for report length] ...

```

8.3 APPENDIX C. SOURCE CODE (Page 72 of 78)

```
# === FILE: templates/analytics/dashboard.html ===
{% extends "analytics/dashboard_base.html" %}

{% block dashboard_content %}
<!-- Section Header -->
<div class="section-header">
    <h1 class="section-title">
        <svg xmlns="http://www.w3.org/2000/svg" width="28" height="28"
fill="var(--primary-color)" viewBox="0 0 16 16">
            <path d="M8 4a.5.5 0 0 1 .5.5V6a.5.5 0 0 1-1 0V4.5A.5.5 0 0
1 8 4zM3.732 5.732a.5.5 0 0 1 .707 0l1.06 1.06a.5.5 0 0
1-.707.708l-1.06-1.06a.5.5 0 0 1 0-.708zM12.268 5.732a.5.5 0 0 1 0
.708l-1.06 1.06a.5.5 0 1 1-.708-.708l1.06-1.06a.5.5 0 0 1 .708 0z"/>
            <path d="M8 1a.5.5 0 0 1 .5.5v.538A3.981 3.981 0 0 1 12 6c0
1.032-.39 1.974-1.03 2.684l-.353.388A7.495 7.495 0 0 1 14.5 13.5a.5.5 0
0 1-1 0A6.495 6.495 0 0 0 8 8.5a6.495 6.495 0 0 0-5.5 5 .5.5 0 0 1-1
0A7.495 7.495 0 0 1 5.383 9.072l-.353-.388A3.981 3.981 0 0 1 4 6a3.981
3.981 0 0 1 3.5-3.962V1.5A.5.5 0 0 1 8 1zM5 6a3 3 0 1 0 6 0 3 3 0 0 0-6
0z"/>
        </svg>
        Overview
    </h1>
    <p class="section-subtitle">Your pet health analytics at a glance</
p>
</div>

<!--          —          Summary          Stat          Cards
----- -->
<div class="row g-3 mb-4">
    <div class="col-6 col-xl-3">
        <div class="stat-card" style="border-left: 4px solid var(--
primary-color);">
            <div class="d-flex align-items-center justify-content-
between mb-2">
```

124
Department of Computer Science

8.3 APPENDIX C. SOURCE CODE (Page 74 of 78)

```

        <div class="stat-card" style="border-left: 4px solid #22C55E;">
            <div class="d-flex align-items-center justify-content-
between mb-2">
                <span class="text-muted small fw-medium text-
uppercase">Assessments</span>
                <div class="bg-light text-success rounded-circle d-flex
align-items-center                                justify-content-center"
style="width:38px;height:38px;">
                    <svg xmlns="http://www.w3.org/2000/svg" width="18"
height="18" fill="currentColor" viewBox="0 0 16 16"><path d="M8 15A7 7 0
1 1 8 1a7 7 0 0 1 0 14zm0 1A8 8 0 1 0 8 0a8 8 0 0 0 0 16z"/><path
d="M10.97 4.97a.235.235 0 0 0-.02.022L7.477 9.417 5.384 7.323a.75.75 0 0
0-1.06 1.06L6.97 11.03a.75.75 0 0 0 1.079-.0213.992-4.99a.75.75 0 0
0-1.071-1.05z"/></svg>
                </div>
            </div>
            <h2 class="fw-bold mb-0">{{ total_assessments }}</h2>
            <small class="text-muted">Health reports</small>
        </div>
    </div>
    <div class="col-6 col-xl-3">
        <div class="stat-card" style="border-left: 4px solid #3B82F6;">
            <div class="d-flex align-items-center justify-content-
between mb-2">
                <span class="text-muted small fw-medium text-
uppercase">Records</span>
                <div class="bg-light text-info rounded-circle d-flex
align-items-center                                justify-content-center"
style="width:38px;height:38px;">
                    <svg xmlns="http://www.w3.org/2000/svg" width="18"
height="18" fill="currentColor" viewBox="0 0 16 16"><path d="M9.5 0a.5.5
0 0 1 .5.5.5.5 0 0 0 .5.5.5.5 0 0 1 .5.5V2a.5.5 0 0 1-.5.5h-5A.5.5 0 0 1
5 2v-.5a.5.5 0 0 1 .5-.5.5.5 0 0 0 .5-.5.5.5 0 0 1 .5-.5h3Z"/><path
d="M3 2.5a.5.5 0 0 1 .5-.5H4a.5.5 0 0 0 0-1h-.5A1.5 1.5 0 0 0 2
2.5v12A1.5 1.5 0 0 0 3.5 16h9a1.5 1.5 0 0 0 1.5-1.5v-12A1.5 1.5 0 0 0
12.5 1h12a.5.5 0 0 0 0 1h.5a.5.5 0 0 1 .5.5v12a.5.5 0 0 1-.5.5h-9a.5.5 0
0 1-.5-.5v-12Z"/></svg>
                </div>
            </div>
            <h2 class="fw-bold mb-0">{{ total_vaccinations|
add:total_medical }}</h2>
            <small class="text-muted">Vaccinations, visits & {{
open_reminders }} open reminders</small>
        </div>
    </div>

```

8.3 APPENDIX C. SOURCE CODE (Page 75 of 78)

```

        </div>
    </div>

    <!--          Quick          Charts          Row
    -->

    <div class="row g-3 mb-4">
        <div class="col-md-5">
            <div class="chart-card">
                <div class="chart-card-title">
                    <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="var(--primary-color)" viewBox="0 0 16 16"><path d="M1
0a1 1 0 0 0-1 1v14a1 1 0 0 0 1 1h14a1 1 0 0 0 1-1V1a1 1 0 0 0-1-1H1zm14
1v14H1V1h14z"/></svg>
                    Species Distribution
                </div>
                {% if species_labels %}
                    <div class="dashboard-chart-wrapper">
                        <canvas id="speciesChart"></canvas>
                    </div>
                {% else %}
                    <div class="empty-state py-3">
                        <p class="mb-0 small">No pets yet. Add a pet to see
this chart.</p>
                    </div>
                {% endif %}
            </div>
        </div>
    </div>

```

8.3 APPENDIX C. SOURCE CODE (Page 76 of 78)

```

<div class="col-md-7">
  <div class="chart-card">
    <div class="chart-card-title">
      <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="var(--accent-color)" viewBox="0 0 16 16"><path d="M4
11H2v3h2v-3zm5-4H7v7h2V7zm5-5h-2v12h2V2zm-2-1a1 1 0 0 0-1 1v12a1 1 0 0 0
1 1h2a1 1 0 0 0 1-1V2a1 1 0 0 0-1-1h-2zM6 7a1 1 0 0 1 1-1h2a1 1 0 0 1 1
1v7a1 1 0 0 1-1 1H7a1 1 0 0 1-1-1V7zm-5 4a1 1 0 0 1 1-1h2a1 1 0 0 1 1
1v3a1 1 0 0 1-1 1H2a1 1 0 0 1-1-1v-3z"/></svg>
      Top Breed Predictions
    </div>
    {% if breed_labels %}
      <div class="dashboard-chart-wrapper">
        <canvas id="breedChart"></canvas>
      </div>
    {% else %}
      <div class="empty-state py-3">
        <p class="mb-0 small">No breed predictions yet.
Identify a breed from a pet page.</p>
      </div>
    {% endif %}
  </div>
</div>

<!--          —          Recent          Activity
----->
<div class="row g-3 mb-4">
  <!-- Recent Predictions -->

```

8.3 APPENDIX C. SOURCE CODE (Page 77 of 78)

```

<div class="col-lg-6">
  <div class="chart-card">
    <div class="chart-card-title">
      <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="var(--accent-color)" viewBox="0 0 16 16"><path
d="M11.742 10.344a6.5 6.5 0 1 0-1.397 1.398h-.001c.03.04.062.078.098.115l3.85 3.85a1 1 0 0 0 0 0-.115-.1zM12 6.5a5.5 5.5 0 1 1-11 0 5.5 5.5 0 0 1 11 0z"/></svg>
      Recent Breed Predictions
    </div>
    {% if recent_predictions %}
      <div class="list-group list-group-flush">
        {% for pred in recent_predictions %}
          <div class="list-group-item border-0 px-0 d-flex
justify-content-between align-items-center">
            <div>
              <strong>{{ pred.pet.name }}</strong>
              <span class="text-muted ms-1">→ {{
pred.predicted_breed }}</span>
            </div>
            <span class="badge bg-primary bg-opacity-10
text-primary rounded-pill px-3 py-2">{{ pred.confidence }}%</span>
          </div>
        {% endfor %}
      </div>
    {% else %}
      <div class="empty-state py-3"><p class="mb-0 small">No
predictions yet.</p></div>
    {% endif %}
  </div>

```


8.3 APPENDIX C. SOURCE CODE (Page 78 of 78)

```

</div>
<!-- Recent Assessments -->
<div class="col-lg-6">
  <div class="chart-card">
    <div class="chart-card-title">
      <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="#22C55E" viewBox="0 0 16 16"><path d="M8 15A7 7 0 1 1
8 1a7 7 0 0 1 0 14zm0 1A8 8 0 1 0 8 0a8 8 0 0 0 0 16z"/><path d="M10.97
4.97a.235.235 0 0 0-.02.022L7.477 9.417 5.384 7.323a.75.75 0 0 0-1.06
1.06L6.97 11.03a.75.75 0 0 0 1.079-.0213.992-4.99a.75.75 0 0
0-1.071-1.05z"/></svg>
      Recent Health Assessments
    </div>
    {% if recent_assessments %}
      <div class="list-group list-group-flush">
        {% for asmt in recent_assessments %}
          <div class="list-group-item border-0 px-0 d-flex
justify-content-between align-items-center">
            <div>
              <strong>{{ asmt.pet.name }}</strong>
              <span class="text-muted ms-1">- {{
asmt.assessment_date|date:"M d, Y" }}</span>
            </div>
            <span class="risk-badge {{
asmt.overall_risk_level }}">{{ asmt.get_overall_risk_level_display }}
Risk</span>
          </div>
        {% endfor %}
      </div>
    ... [code truncated for report length] ...

```

9. BIBLIOGRAPHY

9.1 BIBLIOGRAPHY

The Pet Health and Breed Identification System is an intelligent web application built with Python and Django to assist pet owners in keeping a digital record of their pet profiles, health scans, vaccination history, and medical records. The application enables users to manage pet profiles, execute breed classification models, and view local service providers. The database design enforces relationship integrity through standard normalization constraints. Structured care advisory guidelines provide users with early warning assistance.

Key academic and technical references utilized in the design and development of this project include:

1. Django Software Foundation (2026). Django Web Framework Documentation (v5.2). Retrieved from <https://docs.djangoproject.com/>
2. Google Cloud Vertex AI (2025). AutoML Vision Client Reference Guides for Python. Retrieved from <https://cloud.google.com/vertex-ai/>
3. Keras Team (2025). Image Classification Models and MobileNetV2 Architecture Reference. Retrieved from <https://keras.io/>
4. Python Software Foundation (2026). Python Standard Library and PIL Documentation. Retrieved from <https://www.python.org/>
5. PyMuPDF Development Group (2025). Document Rendering and Story Layout API Reference Guides. Retrieved from <https://pymupdf.readthedocs.io/>
6. SQLite Development Team (2026). SQL Database Management Runtime Architecture and Reference Manuals. Retrieved from <https://www.sqlite.org/>